[CẦN BỔ SUNG: TÊN TRƯỜNG]

[CẦN BỔ SUNG: KHOA / VIỆN]

BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG WEBSITE THƯƠNG MẠI ĐIỆN TỬ TRUNGSHOP KẾT HỢP HỆ THỐNG GỢI Ý SẢN PHẨM DỰA TRÊN KHAI PHÁ LUẬT KẾT HỢP

Sinh viên thực hiện: **Trần Quốc Trung**

Mã số sinh viên: 20200648

Lớp: ICT-01 K65

Giảng viên hướng dẫn: Trần Hải Anh

Học kỳ: 2026.1

2026

LỜI CẢM ƠN

[CẦN BỔ SUNG: sinh viên viết lời cảm ơn theo quá trình thực hiện thực tế; xác nhận tên trường, khoa, giảng viên hướng dẫn và những cá nhân đã hỗ trợ. Không sử dụng nội dung này như một xác nhận về sự tham gia của người chưa được cung cấp thông tin.]

LỜI CAM ĐOAN

[CẦN BỔ SUNG: sinh viên rà soát nguồn mã, quyền sử dụng dữ liệu, nguồn tham khảo và quy định về sử dụng công cụ hỗ trợ của cơ sở đào tạo; tự hoàn thiện, ký và ghi ngày lời cam đoan. Báo cáo này không thay thế cam kết cá nhân của sinh viên.]

TÓM TẮT

Đề tài “Xây dựng website thương mại điện tử TrungShop kết hợp hệ thống gợi ý sản phẩm dựa trên khai phá luật kết hợp” giải quyết hai nhóm công việc: xây dựng quy trình mua bán trực tuyến và khai thác quan hệ đồng xuất hiện giữa các sản phẩm trong đơn hàng để tạo gợi ý có thể giải thích. Hệ thống gồm giao diện khách hàng React, giao diện quản trị React độc lập, API Node.js/Express và cơ sở dữ liệu MongoDB qua Mongoose. Các chức năng được xác định từ mã nguồn gồm tài khoản, danh mục, sản phẩm, tìm kiếm và lọc, giỏ hàng, đặt hàng, theo dõi đơn, quản trị đơn, mã giảm giá, bình luận và thống kê doanh thu.

Luồng gợi ý mặc định chuẩn hóa mỗi đơn thành tập mã sản phẩm, khai phá bằng FP-Growth hoặc Apriori, sinh luật và chọn các luật dạng một sản phẩm suy ra một sản phẩm có lift lớn hơn 1. Kết quả được xếp theo confidence, lift và support; sản phẩm thiếu hoặc hết hàng bị loại. Khi thiếu gợi ý từ luật, hệ thống bổ sung theo danh mục và độ phổ biến, đồng thời gắn nhãn fallback. Giao diện phân biệt “Thường được mua cùng” với “Có thể bạn cũng quan tâm”.

Ba hướng mở rộng được cài đặt và đánh giá trong nhánh nghiên cứu offline: M1 khai phá đa cấp độ, M2 áp dụng ràng buộc nghiệp vụ và cắt tỉa chống đơn điệu, M3 xếp hạng và đa dạng hóa danh mục. Đối chiếu đường gọi cho thấy ba mô hình này chưa được tích hợp vào API mặc định. Kết quả lưu trong repository trên R1/R1b cho thấy chọn tham số bằng validation có tác động lớn hơn các mở rộng còn lại; các cải thiện về recall, độ phủ và đa dạng có sự đánh đổi. Dữ liệu này là dữ liệu giả lập có quy luật cài sẵn, nên chưa đủ để kết luận về doanh thu hoặc hiệu quả với khách hàng thật.

Từ khóa: thương mại điện tử; luật kết hợp; Apriori; FP-Growth; gợi ý sản phẩm; đa cấp độ; ràng buộc; TrungShop.

ABSTRACT

This thesis examines TrungShop, an e-commerce website with an explainable product recommendation component based on association rule mining. The implementation comprises separate React applications for customers and administrators, a Node.js/Express API, and MongoDB accessed through Mongoose. The source code supports accounts, product and category management, search and filtering, carts, order submission and tracking, coupons, comments, and revenue statistics.

The default recommendation pipeline converts orders into sets of product identifiers, mines frequent itemsets using FP-Growth or Apriori, and selects single-antecedent, single-consequent rules with lift greater than one. Candidates are ordered by confidence, lift, and support. Missing and unavailable products are removed. Category-based and popularity-based fallback items fill remaining positions and are explicitly distinguished from association-rule recommendations in both the response and the interface.

Three research extensions are implemented separately: multilevel mining (M1), business constraints and anti-monotone pruning (M2), and ranking with category diversification (M3). Source-level call tracing confirms that these extensions are offline research components rather than part of the default website pipeline. Stored experiments use the synthetic R1 and R1b datasets with embedded purchasing patterns. They show trade-offs between recall, catalog coverage, business-rule compliance, and diversity, with validation-based threshold selection accounting for the largest improvement. These findings do not establish effectiveness on real customers or causal business impact. Remaining verification and institutional metadata are explicitly identified.

Keywords: e-commerce; association rules; Apriori; FP-Growth; explainable recommendations; multilevel mining; constraints.

MỤC LỤC

LỜI CẢM ƠN 1

LỜI CAM ĐOAN 2

TÓM TẮT 3

ABSTRACT 4

MỤC LỤC 5

DANH MỤC HÌNH 8

DANH MỤC BẢNG 9

DANH MỤC TỪ VIẾT TẮT 10

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 11

1.1. Lý do chọn đề tài . . . 11
1.2. Mục tiêu nghiên cứu . . . 11
1.3. Đối tượng và phạm vi nghiên cứu . . . 12
1.4. Phương pháp nghiên cứu . . . 12
1.5. Công nghệ sử dụng . . . 13
1.6. Kết quả dự kiến và đóng góp của đề tài . . . 13
1.7. Bố cục báo cáo . . . 14

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 15

2.1. Tổng quan thương mại điện tử . . . 15
2.2. Hệ thống gợi ý sản phẩm . . . 15
2.3. Các phương pháp xây dựng hệ gợi ý . . . 16
2.4. Khai phá dữ liệu . . . 16
2.5. Khai phá luật kết hợp . . . 17
2.6. Tập mục phổ biến – Frequent Itemset . . . 17
2.7. Thuật toán Apriori . . . 18
2.8. Thuật toán FP-Growth . . . 18
2.9. Các độ đo đánh giá luật kết hợp . . . 19
2.9.1. Support . . . 19
2.9.2. Confidence . . . 20
2.9.3. Lift . . . 20
2.9.4. Kulczynski . . . 20
2.9.5. Imbalance Ratio . . . 20
2.9.6. So sánh và điều kiện diễn giải . . . 21
2.10. Khai phá luật kết hợp đa cấp độ . . . 21
2.11. Khai phá luật kết hợp dựa trên ràng buộc . . . 22
2.12. Xếp hạng và giảm trùng lặp kết quả gợi ý . . . 22

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 24

3.1. Khảo sát bài toán . . . 24
3.2. Yêu cầu chức năng . . . 24
3.3. Yêu cầu phi chức năng . . . 25

3.4. Đối tượng sử dụng hệ thống . . . 26
3.5. Phân tích nghiệp vụ . . . 26
3.6. Kiến trúc tổng thể . . . 29
3.7. Thiết kế cơ sở dữ liệu MongoDB . . . 30
3.8. Phân tích các collection và quan hệ . . . 30
3.9. Thiết kế API . . . 31
3.10. Thiết kế quy trình gợi ý sản phẩm . . . 32

CHƯƠNG 4. XÂY DỰNG WEBSITE TRUNGSHOP 35
4.1. Môi trường và công nghệ phát triển . . . 35
4.2. Xây dựng frontend React . . . 35
4.3. Xây dựng backend Node.js và Express . . . 36
4.4. Thiết kế và triển khai MongoDB . . . 36
4.5. Chức năng tài khoản và xác thực . . . 37
4.6. Quản lý sản phẩm và danh mục . . . 38
4.7. Tìm kiếm và lọc sản phẩm . . . 38
4.8. Chức năng giỏ hàng . . . 39
4.9. Đặt hàng và quản lý đơn hàng . . . 39
4.10. Chức năng quản trị . . . 40
4.11. Tích hợp hệ thống gợi ý . . . 40

CHƯƠNG 5. XÂY DỰNG HỆ THỐNG GỢI Ý SẢN PHẨM 42
5.1. Phát biểu bài toán gợi ý sản phẩm . . . 42
5.2. Dữ liệu đầu vào . . . 42
5.3. Tiền xử lý dữ liệu đơn hàng . . . 43
5.4. Biểu diễn giao dịch . . . 43
5.5. Khai phá tập phổ biến . . . 44
5.6. Sinh luật kết hợp . . . 45
5.7. Các chỉ số đánh giá luật trong hiện thực . . . 45
5.8. Kiến trúc module mining và recommendation . . . 46
5.9. M1 – Khai phá luật kết hợp đa cấp độ . . . 47
5.9.1. Hierarchy và quan hệ sản phẩm–danh mục . . . 47
5.9.2. Support và khai phá theo cấp . . . 48
5.9.3. Loại luật tầm thường và phát hiện dư thừa . . . 48
5.9.4. Ánh xạ luật danh mục về sản phẩm . . . 49
5.9.5. Ví dụ kiểm thử nhỏ có thể tái hiện . . . 49
5.9.6. Ví dụ từ danh mục TrungShop và giới hạn . . . 50
5.10. M2 – Khai phá dựa trên ràng buộc . . . 51
5.10.1. Ràng buộc nghiệp vụ và dữ liệu sử dụng . . . 51
5.10.2. Push và post trong Apriori, FP-Growth . . . 52
5.10.3. Ví dụ xử lý và kết quả . . . 52
5.10.4. Ưu điểm và hạn chế . . . 53
5.11. M3 – Xếp hạng và đa dạng hóa kết quả . . . 53
5.11.1. Nguồn ứng viên và chiến lược . . . 53
5.11.2. Công thức và nguồn gốc . . . 54
5.11.3. Đa dạng hóa theo danh mục . . . 55
5.11.4. Ví dụ chạy trên dữ liệu TrungShop . . . 55
5.11.5. Ưu điểm, hạn chế và vị trí học thuật . . . 56
5.12. Quy trình gợi ý tổng thể . . . 56

5.13. Phân tích độ phức tạp . . . 58
5.14. Ví dụ chạy xuyên suốt từ đơn hàng đến gợi ý . . . 58

CHƯƠNG 6. THỰC NGHIỆM VÀ ĐÁNH GIÁ 60
6.1. Môi trường thực nghiệm . . . 60
6.2. Dữ liệu thực nghiệm . . . 60
6.3. Cách sinh dữ liệu đơn hàng . . . 61
6.4. Thiết lập tham số và chống rò rỉ . . . 62
6.5. Kịch bản kiểm thử . . . 62
6.6. Kiểm thử chức năng website . . . 63
6.7. Kiểm thử thuật toán khai phá . . . 64
6.8. So sánh Apriori và FP-Growth . . . 64
6.9. Đánh giá M1 . . . 65
6.10. Đánh giá M2 . . . 66
6.11. Đánh giá M3 . . . 67
6.12. Thời gian chạy và tài nguyên . . . 68
6.13. Phân tích kết quả . . . 69
6.14. Hạn chế của thực nghiệm . . . 70

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 71
7.1. Kết quả đạt được . . . 71
7.2. Những đóng góp của đồ án . . . 71
7.3. Những hạn chế . . . 72
7.4. Hướng phát triển . . . 72

TÀI LIỆU THAM KHẢO 73

PHỤ LỤC A. BẢN ĐỒ MÃ NGUỒN 75

PHỤ LỤC B. DANH SÁCH ROUTE THEO ROUTER 76

PHỤ LỤC C. NHẬT KÝ KIỂM CHỨNG 80

PHỤ LỤC D. CÁC BƯỚC TÁI LẬP 81

PHỤ LỤC E. DỮ LIỆU VÍ DỤ MỚI 82
E.1. Samsung Galaxy S24 FE 128GB . . . 82
E.2. iPhone 15 128GB . . . 83

DANH MỤC HÌNH

Hình 3.1. Sơ đồ use case theo các quyền hiện có; quan hệ nối biểu thị quyền sử dụng. . . 27
Hình 3.2. Luồng xử lý đặt hàng theo controller `order.js`. 28
Hình 3.3. Máy trạng thái đơn hàng từ `TRANSITIONS`. 28
Hình 3.4. Kiến trúc TrungShop và ranh giới nghiên cứu offline. 29
Hình 3.5. Sơ đồ dữ liệu MongoDB; đường đứt chỉ liên kết logic, hộp con biểu thị dữ liệu nhúng. 31
Hình 3.6. Luồng gợi ý mặc định đã tích hợp trong trang chi tiết. 33
Hình 5.1. Nguyên tắc mở rộng giao dịch M1 từ quan hệ category_id. 47
Hình 5.2. Quy trình thực nghiệm M1–M3 và ranh giới train/validation/test. 57
Hình 6.1. Thời gian một cấp R1/R1b theo Bảng 6.3; biểu đồ tái vẽ từ số liệu đã lưu. . . . 65
Hình 6.2. Recall và coverage trên R1b; số liệu đã lưu, không phải đo mới. 68

DANH MỤC BẢNG

Bảng 0.1. Các từ viết tắt sử dụng trong báo cáo. 10
Bảng 1.1. Công nghệ khai báo và vai trò trong hệ thống. 13
Bảng 2.1. Phân loại phương pháp gợi ý và phạm vi hiện thực. 16
Bảng 3.1. Chức năng và trạng thái theo mã nguồn. 24
Bảng 3.2. Các collection chính theo Mongoose Model. 30
Bảng 3.3. API tiêu biểu đã đối chiếu router. 31
Bảng 5.1. Phân rã bài toán theo các khối được gọi trong mã. 42
Bảng 5.2. Hợp đồng của các khối chính trong luồng tích hợp. 46
Bảng 5.3. Ràng buộc M2-A và M2-B trong `business.js`. 51
Bảng 5.4. Chiến lược `scoreOf` và `rankCandidates`. 54
Bảng 6.1. Phân biệt dữ liệu demo, dữ liệu nghiên cứu và dữ liệu benchmark. 60
Bảng 6.2. Kịch bản được đối chiếu với bộ test hiện có. 62
Bảng 6.3. Benchmark một cấp trên R1/R1b đã lưu trong repository. 64
Bảng 6.4. So sánh M1 với mặc định và baseline đã chọn ngưỡng. 65
Bảng 6.5. M2 cải thiện tuân thủ ràng buộc nhưng giảm độ phủ catalog. 66
Bảng 6.6. Tác động xếp hạng và tiêu chí chọn tham số M3. 67
Bảng A.1. Bản đồ module trọng tâm. 75
Bảng B.1. Route khai báo trong các tệp router. 76
Bảng E.1. Đầu ra minh họa Samsung Galaxy S24 FE 128GB. 82
Bảng E.2. Đầu ra minh họa iPhone 15 128GB. 83

DANH MỤC TỪ VIẾT TẮT

Bảng 0.1. Các từ viết tắt sử dụng trong báo cáo.

Từ	Diễn giải
API	Application Programming Interface – giao diện lập trình ứng dụng
ARM	Association Rule Mining – khai phá luật kết hợp
CRUD	Create, Read, Update, Delete
FP	Frequent Pattern – mẫu phổ biến
IR	Imbalance Ratio – tỷ số mất cân bằng
JWT	JSON Web Token
Kulc	Kulczynski – trung bình hai xác suất có điều kiện
MMR	Maximal Marginal Relevance; ở đây chỉ dùng để mô tả ý tưởng tham lam
ODM	Object Document Mapper; Mongoose ánh xạ tài liệu MongoDB
PRNG	Pseudorandom Number Generator – bộ sinh số giả ngẫu nhiên
RSS	Resident Set Size – bộ nhớ thường trú của tiến trình
SD	Standard Deviation – độ lệch chuẩn
TTL	Time To Live – thời gian hiệu lực của cache
UI	User Interface – giao diện người dùng

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Một cửa hàng trực tuyến cần giúp người dùng tìm sản phẩm và hoàn thành giao dịch với số bước hợp lý. Khi danh mục có nhiều điện thoại và phụ kiện, người dùng thường biết sản phẩm chính nhưng chưa biết các sản phẩm thường được mua cùng. Chỉ hiển thị các mặt hàng cùng danh mục có thể tạo danh sách gồm các sản phẩm thay thế, trong khi nhu cầu mua kèm thường là các sản phẩm bổ trợ như ốp lưng, kính bảo vệ hoặc sạc. Dữ liệu đơn hàng cung cấp một dấu vết trực tiếp của hành vi đồng mua mà có thể khai thác mà không yêu cầu khách hàng đánh giá sao.

Luật kết hợp phù hợp với bài toán vì đầu vào có thể chỉ là các tập sản phẩm trong đơn hàng. Luật $A \Rightarrow B$ cho biết trong lịch sử, các giao dịch có A thường cũng có B với một mức độ đo được. Khác với một điểm dự đoán khó giải thích, luật có thể đi kèm số đơn đồng xuất hiện, confidence và lift. Đây là lợi thế khi người phát triển cần kiểm tra tại sao một sản phẩm được gợi ý và phân biệt tín hiệu dữ liệu với phương án dự phòng.

Tuy vậy, đồng xuất hiện không chứng minh quan hệ nhân quả, sự tương thích kỹ thuật hoặc nhu cầu của từng cá nhân. Một sản phẩm bán chạy có thể đồng xuất hiện với nhiều sản phẩm chỉ do phổ biến. Danh sách cùng danh mục có thể không phù hợp với mục tiêu bán phụ kiện. Đề tài vì thế nghiên cứu cả khai phá mẫu, kiểm soát chất lượng luật, ràng buộc nghiệp vụ và xếp hạng; đồng thời giữ ranh giới giữa kết quả đã tích hợp và kết quả thực nghiệm.

1.2. Mục tiêu nghiên cứu

Mục tiêu phần mềm là khảo sát và mô tả đúng một website có chu trình từ duyệt hàng đến lưu đơn, xác thực người dùng, quản trị dữ liệu và hiển thị gợi ý mua kèm. Mục tiêu thuật toán là trình bày, kiểm chứng hai cách khai phá tập phổ biến cùng cơ chế sinh luật và đo lường quan hệ. Mục tiêu thực nghiệm là đánh giá ảnh hưởng của ngưỡng, phân cấp, ràng buộc và thứ tự ứng viên trong một giao thức tránh sử dụng dữ liệu tương lai để học.

Báo cáo không đặt mục tiêu chứng minh tăng doanh thu do chưa có thử nghiệm kinh

doanh đối chứng. Mục tiêu có thể kiểm tra được là tính đúng của đầu ra khai phá, quan hệ giữa API và UI, cách lọc ứng viên, khả năng tái lập dữ liệu giả lập, và mức đánh đổi của các mô hình theo các chỉ số offline.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng là mã nguồn trong repository `Product-Recommendation`, tập trung vào `Shop/Shop/backendapi`, `Shop/Shop/frontend` và `Admin/Admin`. Trong các chương sau, đường dẫn bắt đầu bằng `src/`, `research/`, `scripts/` hoặc `test/` được hiểu tương đối với thư mục backend, trừ khi ghi rõ Shop hoặc Admin. Các tên tệp và hàm được giữ theo mã nguồn, kể cả cách viết `ultils` trong frontend.

Phạm vi tích hợp gồm luật kết hợp một cấp trong trang chi tiết sản phẩm. M1–M3 nằm trong `research/`, phục vụ phân tích offline. Dữ liệu thực nghiệm R1 có 41 sản phẩm, R1b có 48 sản phẩm, trong đó 7 sản phẩm bổ sung để khảo sát tình huống ít giao dịch. Đây là danh mục của bộ sinh nghiên cứu, không phải kiểm kê số sản phẩm đang có trong một cơ sở dữ liệu triển khai.

Không tìm thấy trong luồng được khảo sát cơ chế thanh toán qua cổng ngân hàng, quản lý tồn kho định lượng, vận chuyển tích hợp nhà cung cấp hay cá nhân hóa bằng hồ sơ sở thích. Báo cáo không coi những nội dung này là chức năng đã hoàn thiện. Việc có các dependency liên quan trong `package.json` cũng không đủ chứng minh một thuật toán hoặc dịch vụ được gọi thực tế.

1.4. Phương pháp nghiên cứu

Phương pháp thứ nhất là phân tích tĩnh: đọc cấu trúc dự án, router, controller, middleware, Mongoose Model, dịch vụ và component; lần theo import và lời gọi từ trang chi tiết đến API và thuật toán. Phương pháp này xác định được các đường gọi và điều kiện trong mã, nhưng không tự xác nhận trạng thái của hệ thống production.

Phương pháp thứ hai là kiểm chứng bằng mã thực thi offline: sử dụng các hàm thuần để tái sinh dữ liệu từ seed, đối chiếu checksum, chạy ví dụ truy vấn và kiểm thử thuật toán. Kết quả mới được tách khỏi các benchmark đã lưu từ môi trường trước đó. Kiểm thử có model giả được phân biệt với kiểm thử MongoDB thật và kiểm thử trình duyệt.

Phương pháp thứ ba là đối chiếu học thuật với bản PDF *Data Mining: Concepts and Techniques*, ấn bản 3, có trong repository. Chương 6 của sách cung cấp nền tảng khai phá mẫu phổ biến và các độ đo; Chương 7 trình bày khai phá nâng cao, gồm đa cấp, ràng buộc và top-k giảm dư thừa. Các công thức do đồ án phối hợp thêm được ghi rõ nguồn gốc để tránh gán nhầm cho giáo trình.

1.5. Công nghệ sử dụng

Bảng 1.1. Công nghệ khai báo và vai trò trong hệ thống.

Thành phần	Công nghệ được khai báo	Vai trò
Shop	React ^18.2.0, React Router 6.4, Redux ^4.2.1, Axios ^1.4.0	Duyệt hàng, tài khoản, giỏ hàng, gợi ý
Admin	React, các trang trong `Admin/Admin/src/pages`	Quản trị sản phẩm, đơn, tài khoản, coupon, bình luận
Backend	Node.js, Express ^4.18.2	Router, kiểm tra dữ liệu, điều phối dịch vụ
Dữ liệu	Mongoose 6.10, MongoDB	Model và truy vấn tài liệu
Xác thực	bcrypt ^5.1.1, jsonwebtoken ^9.0.2	Băm mật khẩu và JWT
Tệp ảnh	multer ^1.4.5-lts.1	Tiếp nhận ảnh có kiểm tra loại và kích thước
Mining	JavaScript tự cài trong `services/mining`	Apriori, FP-Growth và sinh luật

Bảng thể hiện phiên bản khai báo, không khẳng định mọi máy triển khai đều cài đúng một phiên bản đó. Muốn tái lập môi trường cần dùng lockfile và ghi lại phiên bản runtime. Các thư viện `node-apriori` và `node-fpgrowth` xuất hiện trong dependency, nhưng luồng gợi ý hiện tại import thuật toán nội bộ qua `../mining`.

1.6. Kết quả dự kiến và đóng góp của đề tài

Kết quả phần mềm được nhận diện từ mã gồm hai ứng dụng giao diện và một API dùng chung, có phân quyền và nhiều quy tắc bảo toàn lịch sử đơn. Kết quả thuật toán là một module không phụ thuộc MongoDB, có đầu vào/đầu ra thống nhất cho Apriori và FP-Growth, cùng các độ đo luật. Kết quả kiến trúc là cơ chế cache snapshot, cache mô hình và phân biệt nguồn gợi ý để giao diện không trình bày fallback như một phát hiện đồng mua.

Đóng góp nghiên cứu ở mức vận dụng và thực nghiệm: mô hình hóa hierarchy từ cấu trúc Category–Product đang có; xác định những ràng buộc được phép cắt tỉa; khảo sát công thức xếp hạng và đa dạng hóa. Không có cơ sở để tuyên bố đề xuất một thuật toán khai phá hoàn toàn mới. Một đóng góp quan trọng của báo cáo là chỉ ra những giới hạn có thể ảnh hưởng việc bảo vệ đồ án: dữ liệu giả lập, tương thích dựa trên tên, toàn bộ trạng thái đơn cùng đi vào mining và chi phí khai phá đồng bộ.

1.7. Bố cục báo cáo

Chương 2 xây dựng ký hiệu, công thức và nền tảng thuật toán. Chương 3 mô tả yêu cầu và thiết kế suy ra từ mã. Chương 4 phân tích hiện thực website. Chương 5 trình bày luồng gợi ý cùng M1–M3 và ví dụ thực thi. Chương 6 phân biệt kiểm thử mới với kết quả đã lưu và đánh giá giới hạn. Chương 7 tổng hợp kết quả, đóng góp và hướng phát triển. Phụ lục cung cấp bản đồ mã nguồn, API và bằng chứng tái lập.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan thương mại điện tử

Trong phạm vi đồ án, thương mại điện tử là tổ chức quá trình trưng bày sản phẩm, tiếp nhận yêu cầu mua và quản lý trạng thái xử lý đơn qua giao diện web. Một sản phẩm có thông tin mô tả, giá, tình trạng và danh mục; một đơn lưu thông tin người nhận, các dòng hàng và số tiền tại thời điểm tạo. Giữ giá lịch sử của dòng hàng là yêu cầu quan trọng vì giá danh mục có thể đổi sau khi mua.

Cần phân biệt "tiếp nhận đơn" với "thanh toán thành công". Trong TrungShop, API tạo một bản ghi đơn và trạng thái ban đầu `pending`; mã được khảo sát không chứng minh việc thu tiền qua cổng thanh toán. Tương tự, `is_stock` chỉ cho biết trạng thái còn hàng, chưa mô tả số lượng hàng để trừ theo từng đơn.

2.2. Hệ thống gợi ý sản phẩm

Một hệ gợi ý nhận ngữ cảnh và tạo danh sách đối tượng phù hợp. Với trang chi tiết TrungShop, ngữ cảnh mặc định chỉ là mã sản phẩm p đang xem. Đầu ra là danh sách tối đa 5 sản phẩm khác p, có thể truy vết nguồn luật hoặc lý do fallback. Hệ thống không nhận lịch sử duyệt riêng của người dùng trong lời gọi `recommend(productId, query)`; vì vậy đây là gợi ý theo ngữ cảnh sản phẩm, không phải mô hình cá nhân hóa theo từng tài khoản.

Có thể biểu diễn bài toán là $R(p) = \text{TopK}(\mathcal{C}(p))$, trong đó $\mathcal{C}(p)$ là tập ứng viên được sinh từ luật và dữ liệu danh mục. Chất lượng phụ thuộc đồng thời việc sinh ứng viên, thứ tự, lọc khả dụng và điền fallback. Một thuật toán khai phá tốt chưa bảo đảm danh sách cuối hữu ích nếu ứng viên chứa phụ kiện sai dòng máy hoặc fallback chiếm phần lớn kết quả.

2.3. Các phương pháp xây dựng hệ gợi ý

Bảng 2.1. Phân loại phương pháp gợi ý và phạm vi hiện thực.

Phương pháp	Dữ liệu thường cần	Đặc trưng	Quan hệ với TrungShop
Theo nội dung	Thuộc tính sản phẩm, sở thích	So khớp đặc trưng	Chưa thấy mô hình vector nội dung trong luồng mặc định
Lọc cộng tác	Tương tác người–sản phẩm	Học tương đồng/hành vi	Không dùng ma trận user–item trong API hiện tại
Luật kết hợp	Các giao dịch sản phẩm	Học quan hệ đồng xuất hiện, dễ giải thích	Là phương pháp đang tích hợp
Theo tri thức	Ràng buộc chuyên gia	Chọn theo điều kiện nghiệp vụ	M2 nghiên cứu; fallback có dùng tri thức khi ghép M2
Phổ biến	Số đơn chứa sản phẩm	Đơn giản, dùng khi thiếu tín hiệu	Dùng để xếp fallback

Bảng là phân loại lý thuyết, không phải danh sách các thuật toán đều đã được xây dựng. Các phương pháp có thể kết hợp nhưng phải kiểm tra nguồn của mỗi mục đầu ra. Đặc biệt, gợi ý một ốp đúng dòng máy nhờ quy tắc tên có thể rất hữu ích dù không xuất phát từ luật kết hợp.

2.4. Khai phá dữ liệu

Khai phá dữ liệu tìm các mẫu có cấu trúc trong dữ liệu, là một bước trong quy trình gồm lựa chọn, làm sạch, biến đổi, khai phá và đánh giá. Trong bài toán này, chọn trường `items.prd_id` là bước lựa chọn; chuyển về chuỗi, loại trùng và sắp thứ tự là bước biến đổi; tìm tập phổ biến và sinh luật là khai phá; lọc lift, kiểm thử và đánh giá Top-K là đánh giá.

Chuẩn hóa có thể làm thay đổi ngữ nghĩa thống kê. Đếm mỗi sản phẩm một lần trong một đơn phù hợp với support theo giao dịch. Dùng `qty` làm số lần lặp lại sẽ biến bài toán thành đếm số lượng và không còn đúng với công thức support chuẩn. Mã TrungShop chủ động bỏ ảnh hưởng của `qty` trong mining.

2.5. Khai phá luật kết hợp

Cơ sở lý thuyết đối chiếu [1]; các khác biệt hiện thực được chỉ rõ trong phần phân tích mã.

Gọi I là tập mục, $D = \{T_1, \dots, T_N\}$ là tập giao dịch với $T_i \subseteq I$. Luật $A \Rightarrow B$ có $A, B \subseteq I$, $A \neq \emptyset$, $B \neq \emptyset$ và $A \cap B = \emptyset$. Ký hiệu $n(X) = |\{T \in D : X \subseteq T\}|$ là số đếm hỗ trợ. Khi nói một giao dịch chứa $A \cup B$, giao dịch đó phải chứa tất cả mục của cả hai vế; dấu hợp ở đây là hợp tập mục, không phải phép "hoặc" giữa hai biến cố.

Quy trình chuẩn có hai bước: tìm tất cả tập mục có hỗ trợ đạt ngưỡng, sau đó chia mỗi tập phổ biến thành hai vế để tính confidence. Luật là quan hệ có hướng về xác suất có điều kiện; không phải mệnh đề logic đúng tuyệt đối. Confidence 80% không nói rằng 80% người dùng tương lai chắc chắn mua vế phải, vì phân bố có thể thay đổi.

2.6. Tập mục phổ biến – Frequent Itemset

Tập X phổ biến nếu $n(X) \geq \theta$, với θ là ngưỡng số đếm. Khi ngưỡng tỷ lệ là $s \in (0, 1)$, ta dùng $\theta = \lceil sN \rceil$. Trong `resolveMinSupportCount`, số thực nhỏ hơn 1 được hiểu là tỷ lệ; số nguyên từ 1 trở lên được hiểu là số đếm. Do đó truyền `minSupport=1` ở cấp module có nghĩa "ít nhất một đơn", không phải "100% số đơn". API gợi ý giới hạn riêng minSupport trong [0,002; 0,5] nên tránh sự nhập nhằng này ở giao diện HTTP.

Tính chất Apriori phát biểu: nếu X phổ biến thì mọi tập con của X cũng phổ biến; tương đương, tập chứa một tập con không phổ biến không thể phổ biến. Cơ sở là $X \subseteq Y \Rightarrow n(Y) \leq n(X)$. Tính chất này đúng với một ngưỡng chung. Khi ngưỡng thay đổi theo cấp hoặc nhóm, việc kết luận phổ biến giữa các tập cần xét cả ngưỡng riêng, không chỉ support count.

2.7. Thuật toán Apriori

Cơ sở lý thuyết đối chiếu [1]; các khác biệt hiện thực được chỉ rõ trong phần phân tích mã.

Apriori duyệt theo kích thước tập mục. Lần đầu đếm từng mục để tạo L_1. Với mức $k \geq 2$, thuật toán nối các phần tử của L_{k-1} có cùng tiền tố dài $k-2$ để tạo ứng viên, bỏ các ứng viên có tập con không nằm trong L_{k-1}, quét giao dịch để đếm và giữ ứng viên đạt ngưỡng. Khi mức mới rỗng, thuật toán dừng. TrungShop hiện thực các bước này bằng `findFrequent1Itemsets`, `aprioriGen`, `hasInfrequentSubset` và hàm nội bộ `countCandidates`.

Ví dụ sư phạm, không phải dữ liệu đo của cửa hàng: năm giao dịch lần lượt là $\{A, B\}$, $\{A, B, C\}$, $\{A, C\}$, $\{B, C\}$, $\{A, B\}$. Với $\theta = 2$, ta có $L_1 = \{A : 4, B : 4, C : 3\}$. Ba cặp AB, AC, BC có số đếm 3, 2, 2 nên đều phổ biến. Ứng viên ABC được sinh vì mọi cặp con phổ biến nhưng chỉ xuất hiện một lần nên bị loại. Việc qua bước prune chỉ cho phép đếm, không bảo đảm ứng viên chắc chắn phổ biến.

Hiện thực TrungShop so sánh chi phí liệt kê các k-tập con của giao dịch với việc duyệt toàn bộ ứng viên. Giao dịch được thu gọn về các mục có trong ứng viên. Những điều chỉnh này giảm công việc thực tế mà không làm đổi định nghĩa support. Chi phí vẫn có thể tăng rất mạnh khi nhiều mục thường xuyên cùng xuất hiện; số tập con của một tập m mục là 2^m.

2.8. Thuật toán FP-Growth

Cơ sở lý thuyết đối chiếu [1]; các khác biệt hiện thực được chỉ rõ trong phần phân tích mã.

FP-Growth nén các giao dịch vào cây tiền tố. Sau khi đếm hỗ trợ và bỏ mục không phổ biến, các mục trong mỗi giao dịch được xếp theo tần suất giảm dần rồi chèn vào FP-tree. Những giao dịch có tiền tố chung dùng chung nút. Header table lưu mỗi mục, tổng support và chuỗi node-link để truy cập tất cả nút mang mục đó.

Khai phá đi từ mục ít phổ biến trong header. Với một mục hậu tố x, conditional pattern base gồm các đường tiền tố dẫn đến x, có trọng số bằng số đếm ở nút x. Từ cơ sở này

xây cây điều kiện rồi khai phá đệ quy. Nếu cây chỉ có một đường đi, sinh mọi tổ hợp trên đường đó; support của tổ hợp là số đếm nhỏ nhất trong các nút được chọn.

Với năm giao dịch ở mục 2.7, thứ tự ban đầu là A, B, C khi hòa được xử lý ổn định. Cơ sở điều kiện của C gồm đường AB trọng số 1, đường A trọng số 1 và đường B trọng số 1. Từ đó suy ra $n(AC) = 2$ và $n(BC) = 2$, còn $n(ABC) = 1$ bị loại. Kết quả phải trùng Apriori nếu cùng chuẩn hóa và ngưỡng. FP-Growth tránh sinh ứng viên kiểu Apriori nhưng không tránh được bùng nổ số lượng mẫu đầu ra; không thể kết luận luôn nhanh hơn trên mọi dữ liệu.

Trong mã, các khối tương ứng là `FPNode`, `buildFPTree`, `nodeLinks`, `conditionalPatternBase`, `singlePath` và `fpGrowth`. Mỗi giao dịch gốc có trọng số 1; cơ sở điều kiện có trọng số lớn hơn 1. Thứ tự hòa của cây con kế thừa thứ tự cây cha để kết quả có thể đối chiếu với ví dụ giáo trình.

2.9. Các độ đo đánh giá luật kết hợp

Cơ sở lý thuyết đối chiếu [1]; các khác biệt hiện thực được chỉ rõ trong phần phân tích mã.

Dùng một ví dụ tính toán thống nhất, hoàn toàn mang tính minh họa: $N = 10$, $n(A) = 6$, $n(B) = 5$ và $n(A \cup B) = 4$. Các số này mô tả một bảng giao dịch giả định nhỏ nhằm giải thích công thức, không phải benchmark hoặc đơn hàng thật của TrungShop. Các mẫu số phải dương; một luật hợp lệ sinh từ tập phổ biến có cả hai vế xuất hiện nên đáp ứng điều kiện này.

2.9.1. Support

$$\text{supp}(A \Rightarrow B) = \frac{n(A \cup B)}{N}. \qquad (2.1)$$

N là tổng số giao dịch, $n(A \cup B)$ là số giao dịch chứa đồng thời tất cả mục của hai vế. Ví dụ support bằng $4/10 = 0,4 = 40\%$. Support đo độ phổ biến toàn cục, không có hướng. Khi thêm nhiều giao dịch không chứa cả A lẫn B, support giảm dù quan hệ giữa A và B trong các giao dịch liên quan giữ nguyên.

2.9.2. Confidence

$$\text{conf}(A \Rightarrow B) = \frac{n(A \cup B)}{n(A)} = P(B \mid A). \qquad (2.2)$$

$n(A)$ là số giao dịch chứa A. Ví dụ confidence chiều $A \Rightarrow B$ là $4/6 \approx 0,6667$; chiều ngược là $4/5 = 0,8$. Confidence cao có thể chỉ phản ánh B vốn rất phổ biến, vì vậy cần so với $P(B)$ trước khi diễn giải quan hệ.

2.9.3. Lift

$$\text{lift}(A, B) = \frac{\text{supp}(A \cup B)}{\text{supp}(A)\,\text{supp}(B)} = \frac{n(A \cup B)N}{n(A)n(B)}. \qquad (2.3)$$

Ví dụ lift là $4 \times 10/(6 \times 5) = 1,3333$. Giá trị lớn hơn 1 cho thấy đồng xuất hiện nhiều hơn mức kỳ vọng độc lập; bằng 1 tương ứng độc lập theo các xác suất thực nghiệm; nhỏ hơn 1 là ít hơn kỳ vọng. Lift đối xứng và không có chặn trên cố định độc lập với support. Lift cao từ vài đơn rất hiếm có thể thiếu ổn định, vì thế không dùng riêng lift để tuyên bố chất lượng.

2.9.4. Kulczynski

$$\text{Kulc}(A, B) = \frac{1}{2}\left(\frac{n(A \cup B)}{n(A)} + \frac{n(A \cup B)}{n(B)}\right). \qquad (2.4)$$

Ví dụ Kulc bằng $(0,6667+0,8)/2 \approx 0,7333$. Kulc là trung bình confidence hai chiều, nằm trong [0,1] khi dữ liệu hợp lệ. Kulc không phụ thuộc số giao dịch không chứa cả A lẫn B (null-invariant). Tuy nhiên cùng một Kulc có thể đi với mức chênh lệch tần suất hai mục khác nhau; IR giúp đọc thêm thông tin này.

2.9.5. Imbalance Ratio

$$\text{IR}(A, B) = \frac{|n(A) - n(B)|}{n(A) + n(B) - n(A \cup B)}. \qquad (2.5)$$

Mẫu số là số giao dịch chứa A hoặc B, còn tử số là độ chênh support count. Trong ví dụ, IR bằng $|6-5|/(6+5-4) = 1/7 \approx 0,1429$. IR bằng 0 khi hai số đếm bằng nhau; IR cao cho biết hai mục có độ phổ biến mất cân bằng. IR không phải một phép đo trực tiếp "gợi ý sai" và không tự xác định ngưỡng loại luật.

2.9.6. So sánh và điều kiện diễn giải

Nếu giữ nguyên ba số đếm trên nhưng thêm 10 giao dịch không chứa A và B, N tăng thành 20. Confidence, Kulc và IR không đổi; support còn 20% và lift tăng thành $8/3$. Ví dụ cho thấy lift nhạy với null-transactions. Không thể xem lift và Kulc như những thang điểm hoán đổi hoàn toàn; lựa chọn độ đo phải phù hợp mục tiêu và phân bố.

Trong TrungShop, `ruleMeasures(supAB, supA, supB, n)` trả đủ năm độ đo, nhưng phần xếp hạng mặc định chỉ dùng confidence, lift và support. Trường `support` trong itemset là số đếm, còn `support` trong rule là tỷ lệ; nhầm hai cấu trúc này có thể làm sai toàn bộ nhận xét về ngưỡng.

2.10. Khai phá luật kết hợp đa cấp độ

Cơ sở lý thuyết đối chiếu [1]; các khác biệt hiện thực được chỉ rõ trong phần phân tích mã.

Một hierarchy mô tả quan hệ từ khái niệm tổng quát đến cụ thể. Ví dụ phù hợp schema TrungShop là gốc `all` → danh mục Samsung → sản phẩm Galaxy S24 FE. Không có trường danh mục cha trong model `category.js`, do đó không thể tự thêm một tầng "Điện thoại" vào thiết kế cơ sở dữ liệu đã có. Bộ tạo hierarchy hỗ trợ tổng quát nhiều cấp, nhưng phép ánh xạ danh mục thực tế của nghiên cứu sử dụng hai cấp có ý nghĩa.

Mở rộng giao dịch thêm các tổ tiên không gồm gốc. Nếu giao dịch chứa hai sản phẩm cùng danh mục, nút danh mục chỉ được tính một lần. Ngưỡng uniform dùng chung mọi cấp; reduced giảm ngưỡng ở cấp cụ thể; group cho phép ghi đè theo nhóm mục. Đối với tập trộn các nhóm, hiện thực dùng ngưỡng nhỏ nhất trong các mục tham gia, khai phá ở ngưỡng thấp nhất toàn cục rồi lọc theo ngưỡng riêng của tập.

Luật sản phẩm → chính danh mục cha có confidence 1 do định nghĩa taxonomy, không phải phát hiện hành vi mua kèm. Vì vậy phải bỏ tập chứa cặp tổ tiên–hậu duệ. Loại bỏ luật dư thừa do một luật tổng quát dự đoán được là vấn đề khác: cần so support/confidence với giá trị kỳ vọng. TrungShop có hàm `findRedundantRules` cho nghiên cứu nhưng không tự động gọi hàm này trong `mineMultilevel`.

2.11. Khai phá luật kết hợp dựa trên ràng buộc

Cơ sở lý thuyết đối chiếu [1]; các khác biệt hiện thực được chỉ rõ trong phần phân tích mã.

Ràng buộc chống đơn điệu thỏa tính chất: một tập vi phạm thì mọi siêu tập của nó cũng vi phạm. Ví dụ $|S| \leq 2$ hoặc không chứa cặp xung đột; khi đã có ba mục hoặc một cặp xung đột, thêm mục không sửa được vi phạm. Có thể cắt nhánh trước khi đếm hoặc dựng cây điều kiện. Ràng buộc đơn điệu có chiều suy luận khác: nếu tập đã thỏa thì mọi siêu tập thỏa, ví dụ chứa ít nhất một mục thuộc V.

Ràng buộc succinct cho phép mô tả trực tiếp tập mục được phép trước khi đếm; chẳng hạn $S \subseteq V$ cho phép bỏ các mục ngoài V khỏi mọi giao dịch mà vẫn giữ số giao dịch gốc. Ràng buộc convertible như trung bình giá cần thứ tự đặc biệt để có tính chất đơn điệu; hiện thực nghiên cứu không tự triển khai chuyển đổi đó và chỉ hậu lọc. Tổng có thể âm cũng không được tùy tiện coi là chống đơn điệu.

Phân biệt ràng buộc tập với ràng buộc có hướng của luật: yêu cầu sản phẩm gợi ý còn hàng chỉ đặt lên vế phải, không buộc sản phẩm đang xem còn hàng. Đẩy điều kiện này thành "mọi mục trong tập phải còn hàng" sẽ làm mất cả luật có tiền đề hết hàng mà hệ thống vẫn có thể dùng để giới thiệu sản phẩm khác.

2.12. Xếp hạng và giảm trùng lặp kết quả gợi ý

Xếp hạng chọn thứ tự ứng viên theo độ liên quan. Giảm trùng lặp mã sản phẩm bằng Set khác với đa dạng danh mục: một danh sách có năm mã khác nhau vẫn có thể gồm năm điện thoại cùng loại. Mặc định TrungShop chỉ loại trùng mã, còn M3 nghiên cứu thêm phạt danh mục đã xuất hiện.

Hai công thức nghiên cứu là Kulc $(1 - \beta\,\mathrm{IR})$ và conf $P(q)^{-\gamma}$. Công thức thứ hai bằng confidence khi $\gamma = 0$ và bằng lift khi $\gamma = 1$. Đây là cách phối hợp của đồ án; giáo trình cung cấp độ đo thành phần và thảo luận giảm dư thừa ở mức mẫu, không định nghĩa nguyên vẹn hai công thức này cho TrungShop.

Đa dạng hóa chọn tham lam ứng viên tối đa hóa $\lambda\,\mathrm{rel}(i) - (1 - \lambda)\mathbf{1}[c(i)$ đã có$]$. Với $\mathrm{rel}(i) = 1 - \mathrm{rank}(i)/m$ và thứ hạng từ 0, công thức dùng độ liên quan dựa trên thứ

hạng, không dùng trực tiếp thang điểm Kulc hoặc lift. Do đó bước đa dạng có thể đổi thứ tự theo cách khác việc trừ trực tiếp một hằng số khỏi điểm confidence.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Khảo sát bài toán

TrungShop quản lý danh mục điện thoại và phụ kiện, cho phép khách duyệt hàng, xem thông tin, lập giỏ và gửi đơn. Thành viên có thêm hồ sơ, địa chỉ mặc định, lịch sử đơn và quyền bình luận. Quản trị viên làm việc qua một ứng dụng React riêng, nhưng mọi quyền ghi vẫn được kiểm tra ở backend. Tách UI giúp tổ chức nghiệp vụ; nó không thay thế kiểm soát truy cập của API.

Dữ liệu đơn hàng vừa phục vụ vận hành vừa làm nguồn mining. Hai mục đích dùng quy tắc khác nhau: doanh thu chỉ ghi nhận đơn đã giao, trong khi loader gợi ý hiện tại đọc mọi đơn, không lọc trạng thái. Do đó không thể gọi đầu vào mining là “toàn bộ giao dịch mua thành công”. Đây là quyết định hiện thực cần được nêu rõ khi giải thích kết quả và là điểm cần cải tiến khi chuyển sang dữ liệu thật.

3.2. Yêu cầu chức năng

Bảng 3.1 tổng hợp phạm vi có bằng chứng trong router, controller và component. “Có trong mã” thể hiện hiện thực được tìm thấy, không thay cho xác nhận mọi luồng đã chạy trên một bản triển khai thật.

Bảng 3.1. Chức năng và trạng thái theo mã nguồn.

Mã	Chức năng	Bằng chứng chính	Trạng thái
F01	Đăng ký, đăng nhập	`routers/auth.js`, controller `auth.js`	Có trong API và Shop
F02	Hồ sơ, địa chỉ, đổi mật khẩu	`/auth/me`, `pages/Account`	Có trong API và Shop
F03	Danh sách, chi tiết sản phẩm	`product.js`, `ProductBrowser.js`	Có trong API và Shop
F04	Tìm kiếm, lọc giá, danh mục, tồn kho	`libs/product-query.js`	Có trong API và UI
F05	Giỏ hàng, gửi đơn, coupon	reducer `cart.js`, controller `order.js`	Có trong mã
F06	Lịch sử đơn của thành viên	`/order/my`, `Account/Orders.js`	Có trong mã
F07	Bình luận và xóa bình luận	product router, comment router	Thành viên viết; admin quản lý
F08	Quản trị sản phẩm, danh mục	các route create/update/delete	Admin; có quy tắc chặn xóa

Mã	Chức năng	Bằng chứng chính	Trạng thái
F09	Quản trị đơn, chuyển trạng thái	`order-status.js`, `Order/Detail.js`	Admin
F10	Coupon và thống kê	coupon/stats router, Admin pages	Admin
F11	Gợi ý luật một cấp và fallback	`recommendation/service.js`, FBT component	Đã nối vào trang chi tiết
R01	M1 đa cấp độ	`research/multilevel`	Nghiên cứu offline
R02	M2 ràng buộc	`research/constraints`	Nghiên cứu offline
R03	M3 xếp hạng/đa dạng	`research/ranking`	Nghiên cứu offline

Không đồng nhất trạng thái “có mã và test” với “đã hoàn thiện vận hành”. Những phần phụ thuộc cấu hình, database hoặc trình duyệt cần bằng chứng triển khai bổ sung. Các giới hạn còn lại được liệt kê riêng, thay vì dùng một nhãn hoàn thiện chung cho toàn bộ website.

3.3. Yêu cầu phi chức năng

Yêu cầu bảo mật suy ra từ hiện thực là không tin giá hoặc vai trò do client gửi, giới hạn dữ liệu đầu vào, tránh lộ password/hash và kiểm tra chủ thể của tài nguyên. JWT chỉ cung cấp định danh ban đầu; middleware đọc lại tài khoản và role từ DB. CORS giới hạn origin theo cấu hình, các route đăng nhập/đăng ký và preview coupon có rate limiter. Những cơ chế này giảm một số rủi ro, chưa phải chứng nhận bảo mật toàn diện.

Yêu cầu hiệu năng gồm phân trang danh sách, projection bỏ `details` khi tải danh sách sản phẩm, cache mô hình và chỉ đọc trường cần cho mining. Đường lọc giá hiện tính trong bộ nhớ nên chỉ phù hợp quy mô danh mục nhỏ; không có cơ sở đặt cam kết độ trễ hoặc số người dùng đồng thời. [CẦN BỔ SUNG: mục tiêu tải, độ trễ p95, dung lượng dữ liệu, tài nguyên máy và báo cáo kiểm thử tải.]

Yêu cầu khả năng giải thích được hiện thực bằng luật nguồn, số đếm và nhãn fallback. Yêu cầu bảo trì được hỗ trợ nhờ tách thuật toán thuần khỏi Mongoose, tiêm loader vào service và có các bộ kiểm thử. Yêu cầu tái lập nghiên cứu được hỗ trợ bằng seed, phiên bản bộ sinh, checksum, giới hạn tiến trình và lưu kết quả JSON.

3.4. Đối tượng sử dụng hệ thống

Khách chưa đăng nhập có thể xem sản phẩm, gợi ý, bình luận và gửi đơn. API hỗ trợ đặt hàng không tài khoản; tuy nhiên giỏ của khách không được lưu lâu dài bằng hàm `saveCartToStorage` vì hàm này yêu cầu userId. Không nên suy ra rằng khách có một giỏ đồng bộ trên server.

Thành viên có các quyền của khách và thao tác với hồ sơ của chính mình, đổi mật khẩu, xem đơn có `user_id` khớp token, viết bình luận bằng danh tính đã xác thực. Quản trị viên có các trang quản lý, đổi trạng thái và thống kê. Việc cấp mới quyền admin không được thực hiện tùy ý qua API quản lý người dùng; mã hướng đến script `scripts/create-admin.js`.

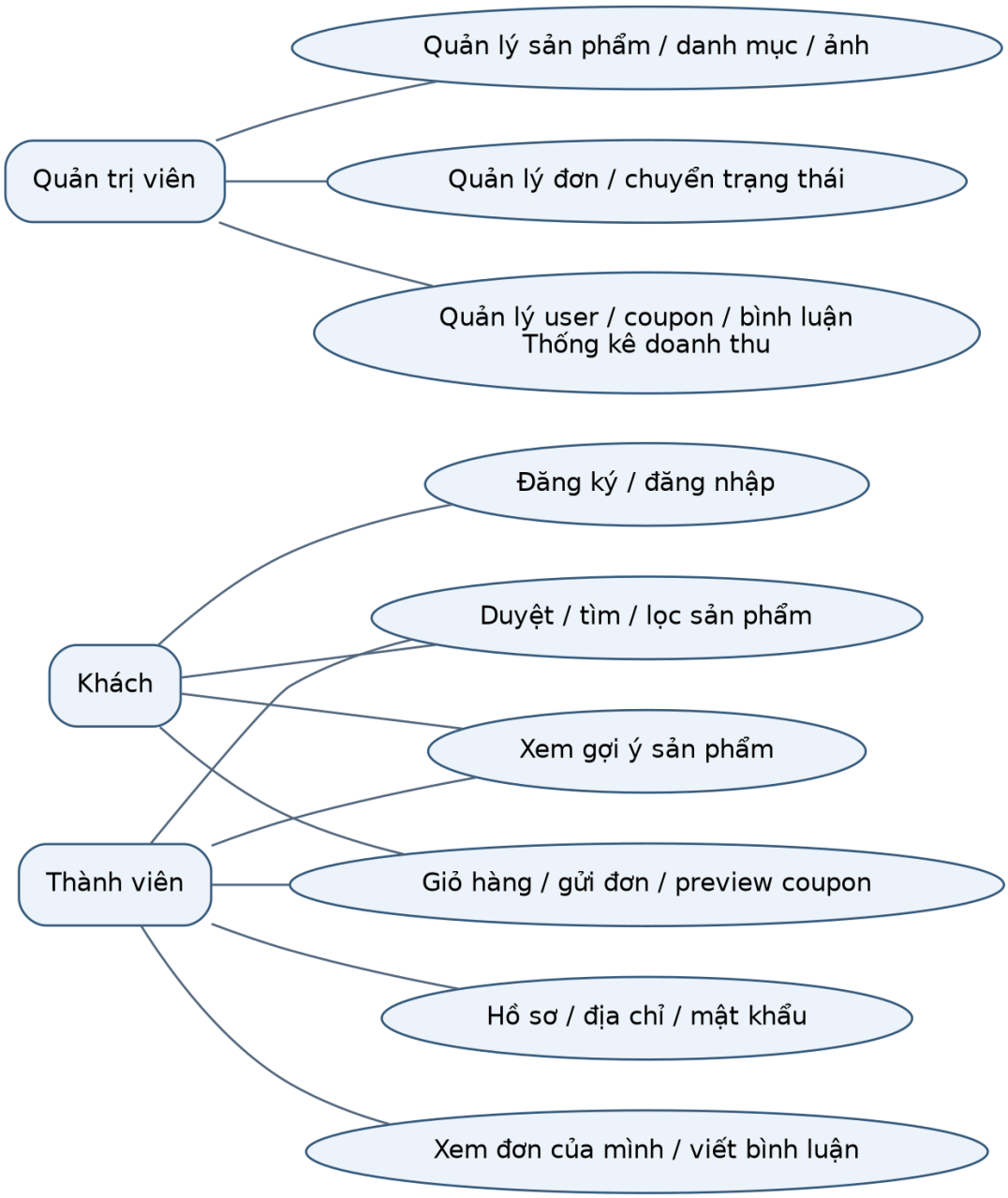
Sơ đồ chỉ gom các use case có route hoặc UI tương ứng. “Đặt hàng” không bao gồm một use case thanh toán trực tuyến chưa được chứng minh. Nghiên cứu M1–M3 không đặt trong quyền của khách hoặc admin vì các script chạy offline bên ngoài website.

3.5. Phân tích nghiệp vụ

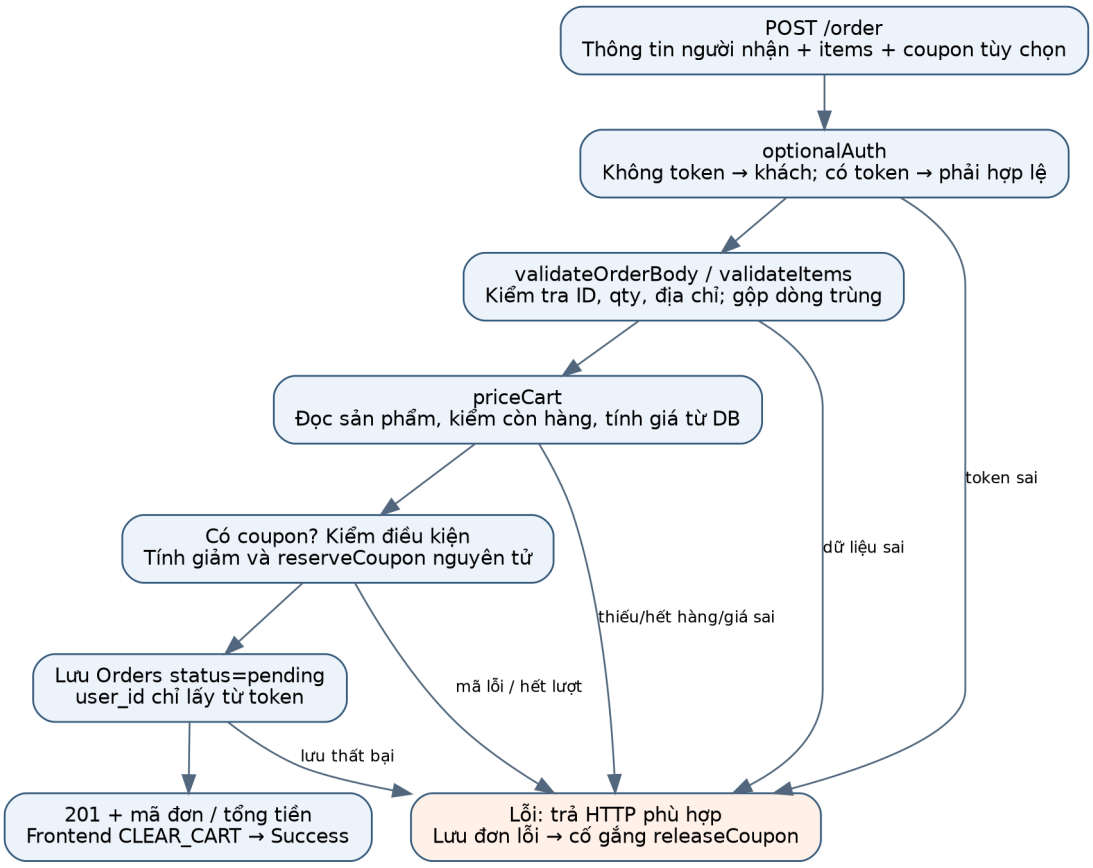
Luồng bán hàng bắt đầu từ chọn sản phẩm và cập nhật giỏ trong trình duyệt. Khi mở giỏ, frontend tải lại thông tin sản phẩm để cập nhật giá và tình trạng. Khi gửi đơn, backend kiểm tra lại toàn bộ đầu vào và tính lại giá từ DB. Nếu có token, `optionalAuth` xác thực và gắn tài khoản; token sai bị từ chối thay vì âm thầm chuyển thành khách.

Mã giảm giá là tài nguyên có thể bị tranh chấp khi nhiều đơn dùng lượt cuối. `reserveCoupon` tăng `used_count` bằng một cập nhật có điều kiện; khi lưu đơn thất bại, `releaseCoupon` cố gắng trả lượt. Đây là bù trừ giữa các thao tác riêng, không phải transaction đa tài liệu. Nếu tiến trình dừng sau giữ lượt nhưng trước lưu đơn, cơ chế hiện tại chưa chứng minh tự phục hồi lượt.

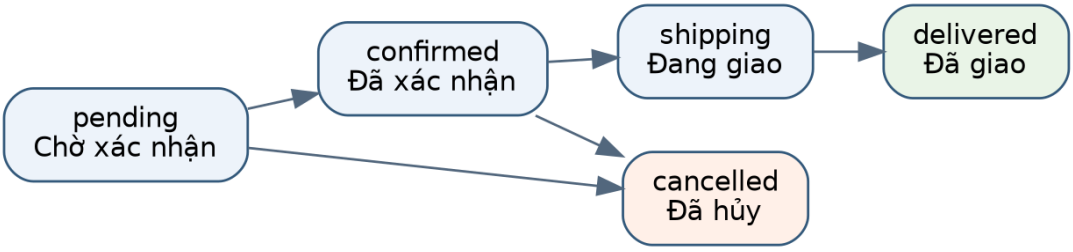
Đơn mới ở `pending`. Admin chỉ được chuyển theo đồ thị quy định; giao hàng thành công ghi `delivered_at`. Đơn cũ thiếu status được diễn giải là pending, không tự cập nhật hàng loạt. Khi hai admin cùng thay đổi, filter trạng thái cũ trong `findOneAndUpdate` giúp một yêu cầu sau có thể nhận 409 thay vì ghi đè im lặng.



Hình 3.1. Sơ đồ use case theo các quyền hiện có; quan hệ nối biểu thị quyền sử dụng.



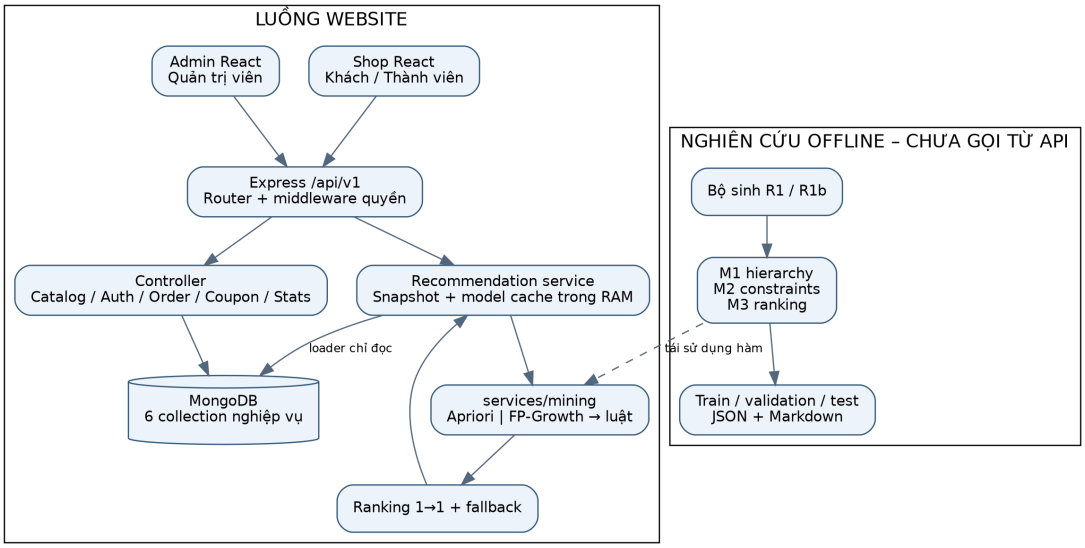
Hình 3.2. Luồng xử lý đặt hàng theo controller `order.js`.



Hình 3.3. Máy trạng thái đơn hàng từ `TRANSITIONS`.

3.6. Kiến trúc tổng thể

Backend là một ứng dụng Express với các nhóm router, controller, model và service, không phải tập microservice độc lập. Các module mang tên `services/mining` và `services/recommendation` chạy trong tiến trình Node của API. Điều này quan trọng đối với độ trễ: mining đồng bộ có thể giữ event loop ngay cả khi các phần đọc DB được gọi bằng Promise.



Hình 3.4. Kiến trúc TrungShop và ranh giới nghiên cứu offline.

Shop và Admin gửi HTTP tới API mặc định `/api/v1`. Express thực hiện CORS, parse body với giới hạn 100 KB, định tuyến, xử lý lỗi JSON và phục vụ tài nguyên tĩnh. Controller phụ trách HTTP và nghiệp vụ; các model truy cập MongoDB. Recommendation service gọi loader để lấy đơn và sản phẩm, rồi dùng module mining và ranking. Nhánh research tái sử dụng các hàm thuật toán nhưng không có đường gọi từ router mặc định vào M1–M3.

3.7. Thiết kế cơ sở dữ liệu MongoDB

Mỗi collection có trường nghiệp vụ và `timestamps`. Quan hệ tham chiếu dùng ObjectId hoặc chuỗi mã; quan hệ nhúng dùng subdocument. Orders nhúng items vì mỗi dòng thuộc vòng đời một đơn và cần lưu giá/qty lịch sử. Địa chỉ giao hàng được chụp vào đơn thay vì luôn đọc địa chỉ hiện tại từ tài khoản; việc người dùng đổi hồ sơ không làm đổi nơi giao của đơn đã gửi.

Bảng 3.2. Các collection chính theo Mongoose Model.

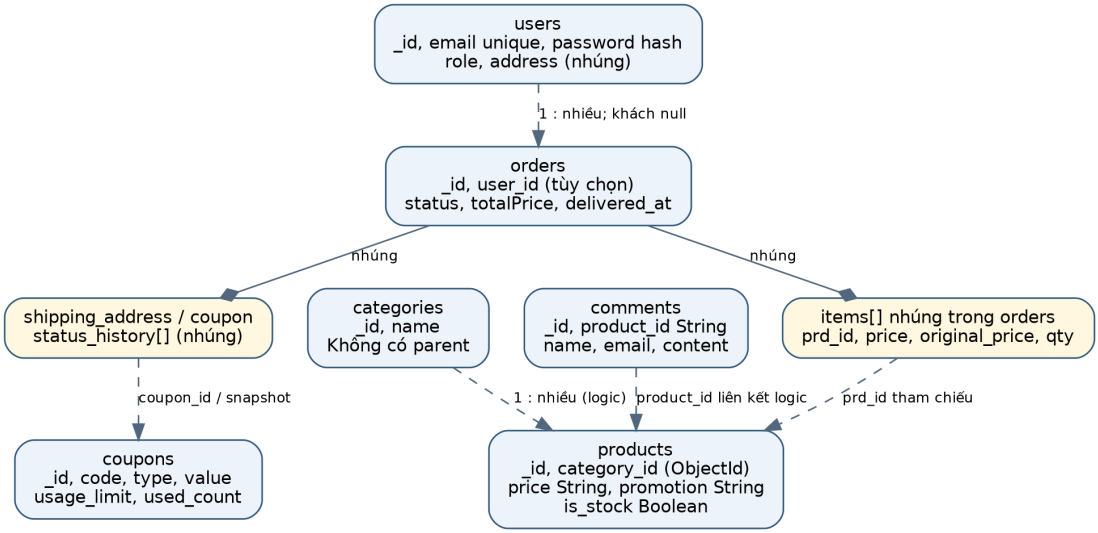
Collection	Thuộc tính nổi bật	Ý nghĩa thiết kế
`categories`	`name`	Danh mục phẳng, không có parent
`products`	`name`, `category_id`, `price`, `promotion`, `is_stock`	Giá và promotion lưu String; trạng thái còn hàng Boolean
`users`	`email`, `password`, `role`, `address`, `password_changed_at`	Email unique; mật khẩu băm; địa chỉ nhúng
`orders`	`user_id`, `items`, `status`, `totalPrice`, `coupon`, `status_history`	Lưu lịch sử giá, coupon và trạng thái
`comments`	`name`, `email`, `content`, `product_id`	Danh tính chụp tại lúc viết; product_id là String
`coupons`	`code`, `type`, `value`, `used_count`, `usage_limit`	Hạn thời gian, hạn lượt và trạng thái hoạt động

Không tự suy ra collection tên `recommendations` hoặc `rules`: service hiện giữ mô hình trong bộ nhớ tiến trình. Khởi động lại làm mất cache; các luật được khai phá lại từ orders khi cần. Các tệp `research-results/*.json` là artifact thực nghiệm, không phải collection production.

3.8. Phân tích các collection và quan hệ

Một category liên quan nhiều product; một user có nhiều order nhưng order của khách cho phép `user_id=null`. Mỗi order có nhiều dòng items; mỗi dòng tham chiếu product và lưu giá/qty. Một product có nhiều comment; comment không tham chiếu user_id mà lưu name/email nên không thể dùng schema hiện tại để khẳng định truy vết mọi bình luận đến một tài khoản bằng khóa ngoại.

Trường `products.category_id` khai báo `ref: "Category"`, trong khi model danh mục được đăng ký tên `Categories`. Đây là bất nhất cần lưu ý nếu sử dụng populate



Hình 3.5. Sơ đồ dữ liệu MongoDB; đường đứt chỉ liên kết logic, hộp con biểu thị dữ liệu nhúng.

tự động. Các luồng đang đọc trực tiếp theo ID hoặc chỉ định model không vì thế mà tự động hỏng; báo cáo không quy kết mọi truy vấn lỗi, nhưng không vẽ quan hệ như một khóa ngoại được DB cưỡng chế.

MongoDB không tự bảo đảm toàn vẹn tham chiếu như khóa ngoại SQL. Controller danh mục chặn xóa khi còn sản phẩm; controller sản phẩm chặn xóa khi đã có đơn, hướng admin chuyển hết hàng. Khi được phép xóa sản phẩm, bình luận liên quan được xóa theo. Các kiểm tra và thao tác là nhiều lệnh riêng, nên cần xem xét race condition nếu nhiều tác nhân ghi đồng thời.

3.9. Thiết kế API

Các đường sau tương đối với `/api/v1` mặc định. Router thể hiện quyền, controller thể hiện kiểm tra nội dung; không nên suy luận quyền chỉ từ tên URL hoặc từ việc nút có hiển thị trong Admin.

Bảng 3.3. API tiêu biểu đã đối chiếu router.

Nhóm	Phương thức và đường dẫn	Quyền	Kết quả chính
Auth	POST `/auth/register`, `/auth/login`	Công khai	Token và user an toàn
Hồ sơ	GET/PUT `/auth/me`; PUT `/auth/me/password`	Thành viên	Hồ sơ hoặc token mới

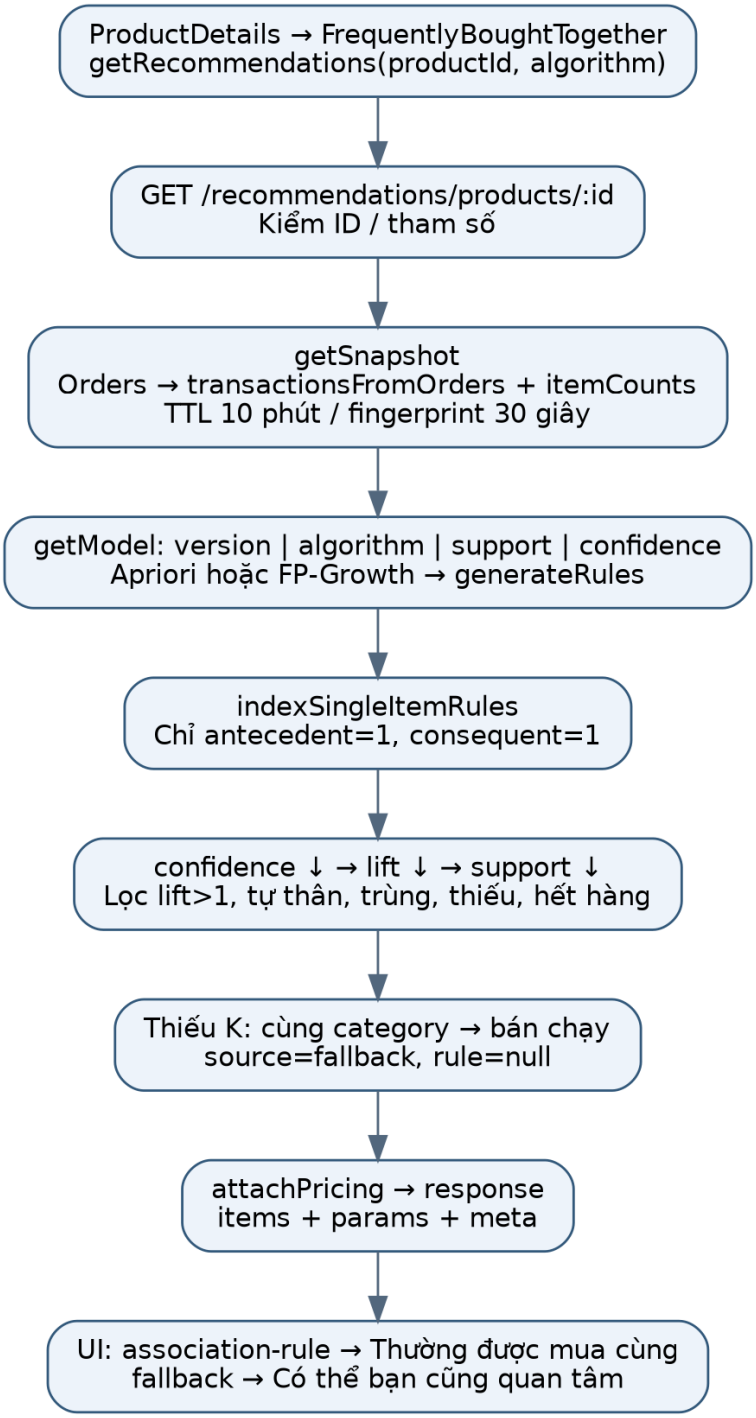
Nhóm	Phương thức và đường dẫn	Quyền	Kết quả chính
Catalog	GET `/products`, `/products/:id`	Công khai	Danh sách phân trang/chi tiết
Danh mục	GET `/categories`, `/categories/:id/products`	Công khai	Danh mục và sản phẩm
Đơn	POST `/order`	Khách hoặc thành viên	Mã đơn và tổng tiền
Đơn cá nhân	GET `/order/my`	Thành viên	Chỉ đơn của token
Đơn quản trị	GET `/order/admin`, `/order/admin/:id`; PATCH status	Admin	Danh sách/chi tiết/trạng thái
Gợi ý	GET `/recommendations/products/:id`	Công khai	`items`, `params`, `meta`
Cache	GET `/recommendations/status`; POST `/recommendations/refresh`	Status công khai; refresh chỉ loopback	Tình trạng hoặc xóa cache
Coupon	POST `/coupons/preview`; `/coupons/admin`	Preview công khai; admin quản lý	Xem trước hoặc dữ liệu coupon
Thống kê	GET `/stats/summary`, `/stats/revenue`, `/stats/top-products`	Admin	Chỉ số và dữ liệu biểu đồ

API gợi ý trả 400 cho mã hoặc tham số sai, 404 khi sản phẩm không tồn tại, 503 khi không đọc được dữ liệu và 500 cho lỗi không dự kiến. Các mã lỗi cần được diễn giải đúng: không tìm thấy luật là tình huống hợp lệ có fallback, không phải cùng nghĩa với lỗi DB. Endpoint refresh kiểm tra `req.socket.remoteAddress`, không dùng role admin; đặt sau reverse proxy cần xem xét kỹ địa chỉ peer và cách giới hạn đường quản trị vận hành.

3.10. Thiết kế quy trình gợi ý sản phẩm

Component `FrequentlyBoughtTogether` gửi productId và algorithm. Router tạo service lười, loader đọc snapshot giao dịch và danh mục. Service chọn mô hình theo snapshot version, thuật toán và ngưỡng. Ranking chỉ lấy luật $1 \to 1$, lọc lift, kiểm tra sản phẩm và điền fallback. Controller có thể gắn lại thông tin giá bán. UI chia theo `source` thành hai khu vực. Tên sản phẩm không phải đầu vào thuật toán khai phá một cấp; ID mới là mục trong giao dịch.

Thiết kế có điểm mạnh là kiểm chứng được nguồn gợi ý và dùng chung mining cho cả



Hình 3.6. Luồng gợi ý mặc định đã tích hợp trong trang chi tiết.

hai thuật toán. Hạn chế là fingerprint chỉ nhìn số đơn và `_id` mới nhất, không nhận biết mọi chỉnh sửa nội dung đơn; sản phẩm được đọc lại mỗi request nhưng dữ liệu luật có thể cũ trong TTL. Không có bằng chứng về cache phân tán hoặc chia sẻ mô hình giữa nhiều tiến trình.

CHƯƠNG 4. XÂY DỰNG WEBSITE TRUNGSHOP

4.1. Môi trường và công nghệ phát triển

Shop được tổ chức theo cấu trúc Create React App với `react-scripts` 5.0.1. Backend khởi động qua `src/bin/www.js`; cấu hình mặc định dùng port 8000 và prefix `/api/v1`, cho phép biến môi trường ghi đè. Hai frontend dùng module `services/Http.js` và `services/Api.js` để đóng gói HTTP. Các dependency và lockfile là cơ sở cài môi trường, còn biến cấu hình bí mật phải do môi trường triển khai cung cấp.

[CẦN BỔ SUNG: phiên bản MongoDB đang triển khai, hệ điều hành máy phát triển của sinh viên, thông số server, URL truy cập bản chạy thật và hướng dẫn triển khai đã xác nhận. Môi trường đo benchmark cũ được trình bày riêng ở Chương 6.]

Không coi cấu hình `.env.example` là thông tin kết nối thật. Báo cáo không trích bí mật từ tệp môi trường hoặc lịch sử Git. Mục tiêu triển khai có thể tái lập là ghi lại runtime, dùng lockfile và giữ riêng thông tin cấu hình khỏi nội dung công khai của đồ án.

4.2. Xây dựng frontend React

`Shop/Shop/frontend/src/App.js` khai báo các trang Home, Category, Search, ProductDetails, Cart, Success, Login, Register và nhóm Account. Layout phân biệt trang mua sắm có cột banner với trang tài khoản dùng chiều rộng đầy đủ. React Router sử dụng các đường dạng `/ProductDetails-:id`, `/Category-:id` và `/account/orders`; báo cáo giữ nguyên kiểu URL thay vì thay bằng một cấu trúc REST tưởng tượng ở frontend.

`ProductBrowser` tập trung logic duyệt catalog. Các component `Product-item`, `PriceTag`, `Pagination` hỗ trợ hiển thị thống nhất. `FrequentlyBoughtTogether` nằm ngay trong trang chi tiết, tự tải gợi ý khi productId hoặc algorithm đổi. Các utility trong `shared/ultils` xử lý giá, token, địa chỉ, nội dung sản phẩm và đường dẫn tài nguyên.

Redux chia trạng thái Auth và Cart. Giỏ hàng được gắn userId khi đăng nhập và tải từ khóa `cart_${userId}`. Đăng xuất xóa giỏ khỏi state nhưng giữ bản lưu của tài khoản; đăng nhập lại có thể phục hồi giỏ đó. Đây là lưu cục bộ theo tài khoản trên một trình

duyệt, không phải giỏ hàng đa thiết bị lưu trong MongoDB.

Các yêu cầu HTTP không đồng bộ có cơ chế bỏ kết quả cũ ở component gợi ý bằng cờ `cancelled`. Khi đổi sản phẩm, gợi ý cũ bị xóa ngay; khi chỉ đổi thuật toán, giao diện có thể giữ và làm mờ danh sách cũ chờ dữ liệu mới. Cờ này ngăn cập nhật state bằng response không còn phù hợp; nó không đồng nghĩa hủy tác vụ mining đã bắt đầu trên server.

4.3. Xây dựng backend Node.js và Express

`src/apps/app.js` cấu hình CORS theo prefix, nhận JSON/urlencoded tối đa 100 KB, gắn `routers/web.js`, trả JSON 404 cho API lạ rồi phục vụ static và xử lý lỗi cuối chuỗi. Các controller async thông thường được bọc `asyncHandler`; recommendation controller có xử lý lỗi riêng. Kiểu tổ chức giúp lỗi validation không cần làm sập tiến trình và hạn chế gửi stack trace về client.

Các lớp có trách nhiệm khác nhau: router đặt URL và middleware quyền; controller kiểm tra request và phối hợp truy vấn; thư viện như pricing, coupon, product-query thực hiện quy tắc dùng lại; service mining xử lý dữ liệu thuần. Việc tách hàm giúp kiểm thử được các quy tắc không cần khởi chạy toàn bộ website.

Một giới hạn của kiến trúc là không có hàng đợi mining trong luồng mặc định. `mine` trả kết quả đồng bộ; bọc trong async function không biến công việc CPU thành tác vụ song song. Khi dữ liệu lớn, một cache miss hoặc nhiều cấu hình ngưỡng mới có thể làm tăng độ trễ toàn bộ API trong cùng tiến trình. Đây là rủi ro có cơ sở từ đường gọi, chưa phải số đo tải thực tế.

4.4. Thiết kế và triển khai MongoDB

Model đặt tên collection tường minh như `products`, `orders` và `users`. Đơn hàng nhúng dòng sản phẩm, địa chỉ và lịch sử trạng thái. Sản phẩm lưu `price` và `promotion` dạng String nên cần một lớp chuyển đổi tiền trước khi so sánh số hoặc tính tổng. `withPricing` cung cấp các trường số đã tính cho API, đồng thời giữ tương thích dữ liệu cũ.

Sơ đồ không giả định transaction, sharding hay replica set khi không có bằng chứng cấu

hình. Việc xử lý coupon có cập nhật nguyên tử trên một tài liệu, còn thao tác giữ lượt và lưu đơn không nằm trong cùng một transaction. Các trường tùy chọn ở order phục vụ khả năng đọc đơn cũ, đặc biệt không tự tính lại giá lịch sử từ giá sản phẩm hiện tại.

[CẦN BỔ SUNG: bản sao dữ liệu đã ẩn thông tin cá nhân và báo cáo index thực tế. Không suy ra số đơn trong DB hiện tại từ con số 1.000 được ghi trong chú thích hoặc từ script seed.]

4.5. Chức năng tài khoản và xác thực

Đăng ký chuẩn hóa email, kiểm tra mật khẩu mới, băm bằng bcrypt với `BCRYPT_ROUNDS=10` và luôn gán role `member`. Trường role do client gửi không được dùng để tự cấp quyền. Đăng nhập chỉ chấp nhận password đã có định dạng hash bcrypt và so sánh bằng bcrypt; tài khoản dữ liệu cũ còn mật khẩu thô không được coi là tương thích tự động.

`signAuthToken` ký HS256 với khóa `JWT_SECRET`, thời hạn `TOKEN_TTL="7d"`. Thiếu hoặc yếu khóa bị kiểm tra qua cấu hình; không có khóa mặc định để đưa vào báo cáo. Middleware `requireAuth` tra lại tài khoản và loại password, `requireAdmin` dùng role hiện tại từ DB. Vì vậy thay đổi role hoặc xóa tài khoản có hiệu lực ở request kế tiếp, không chờ token cũ hết hạn.

Đổi mật khẩu yêu cầu mật khẩu hiện tại đúng, băm mật khẩu mới, lưu `password_changed_at` và cấp token mới. Kiểm tra thu hồi dùng độ phân giải giây của `iat`; điều kiện là token có `iat` nhỏ hơn mốc đã làm tròn. Token được phát cùng giây với thay đổi có thể không bị phân biệt bởi điều kiện này. Cần nêu giới hạn này thay vì khẳng định vô hiệu tuyệt đối mọi token cũ ở độ phân giải nhỏ hơn giây.

Hồ sơ chỉ cho sửa các trường trong `PROFILE_FIELDS`; đổi email yêu cầu mật khẩu hiện tại. API `/auth/me` không nhận ID tài khoản từ URL nên giảm khả năng đọc/sửa hồ sơ người khác bằng cách thay mã. Đơn cá nhân lọc `user_id=req.user._id`, không dùng user_id từ query. Kiểm tra backend là lớp bảo vệ chính ngay cả khi UI có `RequireLogin` hoặc `RequireAdmin`.

4.6. Quản lý sản phẩm và danh mục

Tạo/sửa sản phẩm kiểm tra body, category tồn tại và tên trùng. Các trường `is_stock`, `is_featured` có quy tắc giữ lại giá trị khi update không gửi mới. Chức năng upload nhận trường multipart `image`, tối đa 2 MB, kiểm tra magic bytes cho JPG/PNG/GIF/WEBP và tạo tên ngẫu nhiên phía server. Giới hạn và loại tệp là điều kiện hiện thực, không chỉ dựa vào đuôi tệp từ trình duyệt.

Xóa sản phẩm trước hết đếm đơn có `items.prd_id` tương ứng. Nếu đã có lịch sử, API trả xung đột và yêu cầu chuyển trạng thái hết hàng. Nếu được xóa, controller xóa bình luận cùng sản phẩm. Danh mục có sản phẩm cũng bị chặn xóa. Quy tắc này giữ ổn định lịch sử và hạn chế mất các mục đang tham gia mining.

[CẦN BỔ SUNG: ảnh chụp giao diện quản lý sản phẩm và danh mục từ bản chạy thực tế. Các ảnh sản phẩm trong `public/images` là tài nguyên catalog, không phải ảnh chụp màn hình và không được dùng để giả làm UI đã chạy.]

4.7. Tìm kiếm và lọc sản phẩm

`parseProductQuery` kiểm tra các tham số đơn, không nhận object/array cho từ khóa hoặc giá. Từ khóa tối đa 100 ký tự, tối đa 8 token. `foldVietnamese` chuẩn hóa bỏ dấu để dựng các lớp ký tự dấu cho regex, còn các ký tự khác được escape. Bộ lọc yêu cầu tất cả token xuất hiện trong tên, không phụ thuộc thứ tự; đây không phải tìm kiếm ngữ nghĩa hoặc sửa lỗi chính tả tự động.

Các chế độ sort gồm newest, price_asc, price_desc, name_asc và name_desc. Với truy vấn không liên quan giá, MongoDB thực hiện count, sort, skip và limit; sắp tên sử dụng collation tiếng Việt. Khi cần lọc/sắp theo giá bán, backend đọc danh sách ứng viên đã lọc các thuộc tính khác, chạy `withPricing`, lọc giá số, sắp rồi cắt trang trong bộ nhớ.

Điều này bảo đảm ngữ nghĩa lọc theo giá sau ưu đãi, nhưng chi phí phụ thuộc tổng số ứng viên trước phân trang. Tăng pageSize không phải yếu tố duy nhất của bộ nhớ; một truy vấn giá có thể đọc nhiều bản ghi rồi mới lấy 12 sản phẩm. Hướng cải tiến là chuẩn hóa giá số và có chiến lược index/aggregation phù hợp, cần cân nhắc tương thích dữ liệu cũ.

4.8. Chức năng giỏ hàng

Reducer nhận ADD_TO_CART, UPDATE_CART, DELETE_ITEM_CART, REFRESH_CART_ITEMS và CLEAR_CART. Sản phẩm đã có thì cộng qty; cập nhật và xóa tạo state mới. Khi mở giỏ, `Promise.allSettled` tải lại từng product, chỉ dùng những lần tải thành công để cập nhật giá và trạng thái. Vì request lỗi không chứng minh sản phẩm còn bán, backend vẫn phải kiểm tra cuối cùng khi tạo đơn.

Giỏ của thành viên lưu theo userId trong localStorage. Hàm load có try/catch khi đọc JSON giỏ; tuy nhiên phần khởi tạo `JSON.parse(localStorage.getItem("user"))` ở reducer được gọi trực tiếp. Dữ liệu user lưu sai định dạng có thể cần xử lý bền vững hơn. Không có collection cart trong các model đã khảo sát.

Khi API tạo đơn thành công, frontend dispatch CLEAR_CART và điều hướng Success với mã đơn, tổng tiền. Nếu API lỗi, giỏ được giữ để sửa và gửi lại. Nút gửi có trạng thái chặn gửi lặp ở component, nhưng chưa thấy khóa idempotency phía API để ngăn hai request hợp lệ cùng tạo hai đơn khi mạng chập chờn hoặc client gửi lại chủ động.

4.9. Đặt hàng và quản lý đơn hàng

`validateItems` nhận tối đa 50 dòng; qty mỗi dòng là số nguyên 1–99. Các dòng trùng ID được gộp và chặn tổng ở 99. Backend bỏ giá từ body, đọc sản phẩm bằng `$in`, kiểm tra tồn tại, `is_stock` và giá hợp lệ. `productPricing` chuyển chuỗi tiền sang số nguyên VND; promotion dạng tiền chỉ giảm khi dương và nhỏ hơn giá gốc; promotion dạng chữ là thông tin quà tặng.

Tổng giá gốc bằng tổng original_price × qty; subtotal bằng tổng giá bán × qty; product_discount là hiệu hai tổng. Coupon giảm trên subtotal sau ưu đãi sản phẩm, mỗi đơn tối đa một mã. Tổng cuối bằng max(0, subtotal − coupon_discount). Các giá này được chụp trong order; không tính lại đơn cũ theo catalog mới.

Admin có danh sách phân trang và lọc trạng thái, tìm mã đơn, tên khách, điện thoại hoặc email. Chi tiết giữ được dòng hàng khi không còn thông tin catalog đầy đủ. Trạng thái chỉ đi theo `TRANSITIONS`, không nhảy cóc hoặc quay lui; delivered và cancelled là trạng thái cuối. Không có quyền tự hủy đơn của thành viên được chứng minh trong router;

việc hủy thuộc luồng admin trước giai đoạn shipping.

API không trừ lượng hàng số vì model chỉ có Boolean. Không nên mô tả quy trình như hệ thống quản lý kho có đặt giữ và giải phóng số lượng. Điều kiện kiểm tra `is_stock` cũng chỉ phản ánh dữ liệu tại thời điểm đọc, không phải một cam kết tồn kho được khóa đến khi giao.

4.10. Chức năng quản trị

Admin React có Dashboard, Category, Product, User, Comment, Order và Coupon. `AdminLayout` bao `RequireAdmin`, nhưng API đồng thời áp `requireAdmin` cho từng nhóm ghi và dữ liệu nhạy cảm. Điều này bảo vệ khi người dùng gọi HTTP trực tiếp không qua giao diện.

Thống kê doanh thu chỉ nhận đơn delivered, dùng `totalPrice` đã lưu. Ngày ghi nhận ưu tiên delivered_at, rồi mốc delivered trong status_history; helper còn fallback sang updatedAt khi thiếu hai nguồn trên. Báo cáo phải nêu fallback thực tế này dù chú thích tóm tắt có thể chỉ nói hai nguồn đầu. Thống kê theo múi giờ Việt Nam UTC+7; doanh thu theo sản phẩm dùng giá dòng × qty trước coupon, không phân bổ coupon đơn cho từng sản phẩm.

Quản lý coupon có mã, kiểu fixed/percent, ngưỡng đơn, mức giảm tối đa, hiệu lực, số lượt và trạng thái bật/tắt. Mã được vô hiệu hóa thay vì xóa để giữ lịch sử. Preview không trừ lượt; đặt đơn mới giữ lượt nguyên tử. Vì dữ liệu thống kê và mining có bộ lọc khác nhau, không so trực tiếp số “đơn dùng để gợi ý” với số “đơn ghi nhận doanh thu” mà bỏ qua điều kiện trạng thái.

4.11. Tích hợp hệ thống gợi ý

Trang ProductDetails render `FrequentlyBoughtTogether productId={id}`. Component gọi `getRecommendations`, hàm này gọi đường `recommendations/products/${id}`. Router gắn tại `/recommendations` tạo service từ model Order và Product. Service import `mine` từ module nội bộ và `rankRecommendations` một cấp. Không có import M1–M3 trong chuỗi gọi này.

UI chỉ hiện khu vực có dữ liệu. `source="association-rule"` vào “Thường được mua cùng”; `source="fallback"` vào “Có thể bạn cũng quan tâm”. Chế độ `?research=1` hoặc `REACT_APP_RECO_RESEARCH=true` hiện nút Apriori/FP-Growth và chỉ số; đó là công cụ quan sát thuật toán một cấp, không phải công tắc kích hoạt ba mô hình nghiên cứu nâng cao.

Route cũ `/products/:id/product-suggestions` vẫn tồn tại với controller `products uggestions.js`, nhưng không phải lời gọi của component hiện hành. Controller cũ đặt `minSup=5` rồi so với tỷ lệ cặp trên tổng đơn; với dữ liệu đơn được chuẩn hóa, tỷ lệ không vượt 1 nên điều kiện không đạt. Hơn nữa code cũ không loại trùng mục như module mới. Vì vậy không dùng route cũ để mô tả tính đúng của luồng mới hoặc gọi nó là FP-Growth đầy đủ chỉ dựa trên các đoạn comment.

[CẦN BỔ SUNG: ảnh trang chi tiết ở chế độ khách và chế độ research lấy từ phiên bản chạy thật; xác nhận API URL, dữ liệu và ngày chụp. Báo cáo hiện sử dụng sơ đồ kỹ thuật có nguồn mã, không dựng giao diện giả.]

CHƯƠNG 5. XÂY DỰNG HỆ THỐNG GỢI Ý SẢN PHẨM

5.1. Phát biểu bài toán gợi ý sản phẩm

Đầu vào online gồm mã sản phẩm đang xem p, thuật toán và ngưỡng tùy chọn. Đầu vào học gồm lịch sử orders và catalog products. Đầu ra là một danh sách không quá $K = 5$ sản phẩm, không chứa p, không trùng ID, còn tồn tại và có `is_stock` khác false. Mỗi mục có nguồn `association-rule` hoặc `fallback`; mục từ luật có bằng chứng đồng xuất hiện, mục fallback có lý do riêng.

Mục tiêu của luồng mặc định là gợi ý theo quan hệ mua kèm của sản phẩm, không dự đoán toàn bộ giỏ trong tương lai hoặc cá nhân hóa người dùng. Tiền đề chỉ một sản phẩm là lựa chọn phù hợp với thông tin thực tế có ở trang chi tiết. Sinh và dùng luật nhiều tiền đề trong bối cảnh này sẽ đòi hỏi giả định thêm về các sản phẩm khác mà khách đã chọn; hiện thực chủ động không làm như vậy.

Bảng 5.1. Phân rã bài toán theo các khối được gọi trong mã.

Tầng	Câu hỏi giải quyết	Hiện thực
Dữ liệu	Những mục nào cùng nằm trong một đơn?	`transactionsFromOrders`
Khai phá	Tập nào xuất hiện đủ nhiều?	`apriori` hoặc `fpGrowth`
Luật	Quan hệ có hướng nào đạt confidence?	`generateRules`
Ứng viên	Luật nào phù hợp p đang xem?	`indexSingleItemRules`
Xếp hạng/lọc	Sản phẩm nào ưu tiên và còn bán?	`rankRecommendations`
Dự phòng	Thiếu luật thì điền gì?	Cùng danh mục rồi độ phổ biến
Trình bày	Mục nào thực sự có nguồn luật?	`FrequentlyBoughtTogether`

5.2. Dữ liệu đầu vào

`createModelLoaders` nhận OrderModel và ProductModel qua tham số. `loadOrders` dùng `find({}, {"items.prd_id":1}).lean().exec()`, không lọc status và không lấy danh tính người mua. Việc không lấy thông tin cá nhân là đủ cho thuật toán giao dịch; nó cũng xác nhận hệ thống không cần hồ sơ sở thích cá nhân để sinh luật.

`loadProducts` lấy name, image, price, category_id và is_stock mỗi request. Các trường này phục vụ kiểm tra khả dụng, hiển thị và fallback; nội dung chi tiết sản phẩm không

tham gia khai phá một cấp. Controller đọc thêm price, promotion và is_stock để gắn giá hiển thị qua `attachPricing`.

Fingerprint là chuỗi số đơn và `_id` lớn nhất: `${count}:${latestId}`. Nó rẻ hơn đọc toàn bộ item của tất cả đơn, nhưng không bao phủ mọi thay đổi. Sửa items của một đơn mà giữ nguyên số lượng đơn và ID lớn nhất không làm đổi fingerprint; TTL hoặc invalidate thủ công mới buộc học lại trong trường hợp đó. Đọc đơn và đọc fingerprint được gọi đồng thời cũng không tạo một snapshot transaction nhất quán ở DB.

5.3. Tiền xử lý dữ liệu đơn hàng

`transactionsFromOrders` kiểm tra đầu vào là mảng, ánh xạ mỗi order sang `items .map(item=>item?.prd_id)`, dùng mảng rỗng nếu items thiếu. `normalizeItem` bỏ null/undefined và chuỗi rỗng sau trim; các mục còn lại chuyển bằng String, cho phép ObjectId được sử dụng trực tiếp thông qua toString. `normalizeTransactions` dùng Set để loại trùng và sort để có thứ tự từ điển ổn định.

Đơn rỗng vẫn là một giao dịch và vẫn tham gia mẫu số N. Đây là lựa chọn tường minh trong mã. Nếu dữ liệu bẩn sinh nhiều đơn rỗng, support của mọi sản phẩm giảm; việc tự bỏ đơn rỗng trong báo cáo mà không đổi mã sẽ mô tả sai thuật toán thực tế. Tương tự, không tự lọc đơn cancelled hoặc chỉ giữ delivered khi mã loader hiện đọc tất cả.

Ví dụ minh họa cấu trúc: một đơn có ba dòng `(p, qty=2)`, `(p, qty=1)`, `(q, qty=3)` thành giao dịch `{p,q}`. Support của p và q tăng một đơn, không tăng theo tổng qty. Nếu một order không có mảng items thì giao dịch là `{}`. Ví dụ này giải thích phép biến đổi, không phải trích một đơn khách hàng thật.

5.4. Biểu diễn giao dịch

Sau chuẩn hóa, dữ liệu là mảng các mảng chuỗi đã sắp. Một itemset được biểu diễn `{ items: [...], support: count}`. Hàm `itemsetKey` chuẩn hóa khóa để lập Map tra support; `sortItemsets` tạo thứ tự đầu ra ổn định. Số đếm là số giao dịch chứa tập, không phải tổng số lần mục xuất hiện ở các dòng.

Bất biến cần giữ là mọi item trong một giao dịch xuất hiện nhiều nhất một lần, và mọi

itemset có thứ tự chuẩn. Bước join Apriori dựa vào tiền tố nên thứ tự không chỉ phục vụ dễ đọc. FP-tree dùng thứ tự theo tần suất riêng trong quá trình dựng cây, nhưng đầu ra được sắp lại trước trả kết quả để đối chiếu với Apriori.

Hợp đồng vào/ra thuần JavaScript giúp test không cần MongoDB. Khi đưa dataset nghiên cứu vào module, `history.map(o=>o.items)` đã có dạng giao dịch; không truyền trực tiếp order nghiên cứu cho `transactionsFromOrders` vì định dạng nghiên cứu dùng items là các chuỗi, còn order website dùng các object có prd_id.

5.5. Khai phá tập phổ biến

`mine` chọn hàm từ bảng ALGORITHMS theo tên; mặc định là fpgrowth. Hai thuật toán đều chuẩn hóa đầu vào và dùng chung `resolveMinSupportCount`. Đầu ra gồm itemsets và stats, trong đó Apriori thống kê các mức/candidate, FP-Growth thống kê nút cây, cây điều kiện, single-path shortcut và độ sâu. Những trường stats phục vụ giải thích cấu trúc tính toán; không đồng nhất chúng với thời gian toàn bộ request API.

Ví dụ ở lịch sử 16.000 đơn và minSupport=0,002, số đếm tối thiểu là 32. Một cặp xuất hiện 31 đơn bị loại dù mỗi sản phẩm riêng lẻ có nhiều đơn. Ngược lại, một cặp có 32 đơn có thể tạo luật nhưng vẫn không được gợi ý nếu lift không lớn hơn 1, confidence không đạt ngưỡng hoặc sản phẩm không còn khả dụng.

Ngưỡng thấp tăng số tập và số luật, có thể giúp recall nhưng tăng chi phí và nhiễu. Ngưỡng cao làm mô hình nhỏ, nhưng dễ bỏ quan hệ của sản phẩm ít giao dịch. Đây là động lực của nghiên cứu M1, đồng thời giải thích vì sao so mô hình mở rộng với baseline dùng ngưỡng mặc định mà không có baseline đã chọn ngưỡng sẽ gây kết luận sai về đóng góp thuật toán.

5.6. Sinh luật kết hợp

`generateRules` duyệt các itemset dài ít nhất 2. Với mỗi tập phổ biến L kích thước m, duyệt $2^m - 2$ tập con thực sự khác rỗng để tạo tiền đề A và hậu đề $L \setminus A$. Tra support của cả hai vế trong Map, tính `ruleMeasures`, giữ luật đạt minConfidence với dung sai 10^{-12}. Kết quả được sắp confidence rồi lift, sau đó thứ tự tập mục để ổn định.

Nếu danh sách itemsets thiếu support của một tập con cần thiết, hàm ném lỗi thay vì dùng 0 hoặc suy đoán. Tính đầy đủ của tập phổ biến là tiền điều kiện quan trọng. Các bước lọc itemset nâng cao phải giữ hoặc cung cấp các support cần thiết trước khi sinh luật; lọc mọi singleton thấp ngưỡng riêng quá sớm trong M1 có thể làm hỏng điều kiện này.

Hàm dùng bitmask `1 << n` và từ chối itemset dài hơn 30. Giới hạn 30 bảo vệ biểu diễn bitwise, chưa phải bảo đảm hiệu năng: $2^{30} - 2$ trường hợp vẫn quá lớn trong thực tế. Luồng mặc định khai phá và sinh mọi luật trước khi chỉ mục $1 \to 1$ loại phần không dùng; hướng tối ưu là giới hạn sinh luật cần thiết, như các mô hình offline đã làm với `maxRul eItemsetSize=2`.

5.7. Các chỉ số đánh giá luật trong hiện thực

`ruleMeasures` trả `support`, `supportCount`, `confidence`, `lift`, `kulczynski` và `imbal anceRatio`. Công thức đúng theo mục 2.9 khi số đếm đầu vào hợp lệ. Hàm tự nó không là một bộ xác minh bảng xác suất: lời gọi thủ công với supAB lớn hơn supA/supB hoặc mẫu số 0 có thể tạo giá trị vô nghĩa. Trong quy trình chuẩn, các số đếm lấy từ tập phổ biến đầy đủ nên quan hệ đếm được bảo đảm bởi miner.

`indexSingleItemRules` chỉ đưa luật có antecedent.length=1 và consequent.length=1 vào chỉ mục theo productId. `rankRecommendations` dùng điều kiện `rule.lift > min Lift`, mặc định minLift=1, nghĩa là lift đúng 1 bị loại. Không có tham số minLift công khai trong `parseOptions`; không nên liệt kê nó như query HTTP được hỗ trợ mặc định.

Sau sắp xếp, thuật toán bỏ chính sản phẩm xem, sản phẩm trùng, không tìm thấy và hết hàng, đồng thời đếm skipped theo nhóm. `matchedRuleCount` là số luật khớp tiền đề trước khi lọc và cắt limit, không phải số sản phẩm cuối được hiển thị. Việc diễn giải metadata đúng giúp tránh nhận xét "API trả 100 gợi ý" chỉ vì matchedRuleCount bằng

100.

5.8. Kiến trúc module mining và recommendation

Bảng 5.2. Hợp đồng của các khối chính trong luồng tích hợp.

Module/hàm	Đầu vào	Đầu ra	Trách nhiệm
`transactionsFromOrders`	Order `items[{prd_id}]`	Mảng giao dịch	Chuẩn hóa, loại trùng
`apriori`, `fpGrowth`	Giao dịch và minSupport	itemsets, stats	Đếm tập phổ biến
`generateRules`	itemsets, N, minConfidence	Danh sách luật	Sinh hai vế và độ đo
`createModelLoaders`	Model Mongoose	Ba loader	Tách DB khỏi thuật toán
`createRecommendationService`	Loader và cấu hình cache	recommend/status/invalidate/getModel	Điều phối và cache
`rankRecommendations`	p, ruleIndex, catalog, itemCounts	items, skipped, matchedRuleCount	Chọn, lọc và fallback
`attachPricing`	Dữ liệu gợi ý, loader giá	Dữ liệu có giá bán	Bổ sung thông tin hiển thị

Snapshot gồm transactions, itemCounts, fingerprint, loadedAt và version. Mô hình có khóa version|algorithm|minSupport|minConfidence. Limit không nằm trong khóa vì nó chỉ ảnh hưởng cắt danh sách, không thay đổi tập luật. Snapshot dùng chung cho hai thuật toán; mỗi cấu hình mining dùng mô hình riêng. TTL mặc định là 10 phút, khoảng kiểm fingerprint 30 giây.

Các request cùng chờ snapshotPromise hoặc promise của một mô hình để tránh lặp tải/khai phá cho cùng khóa. Tuy nhiên mining thực hiện đồng bộ trên CPU; Promise chỉ điều phối kết quả, không tạo worker thread. Số tổ hợp ngưỡng là số thực nên Map mô hình có thể tăng nếu nhiều request chọn các ngưỡng khác nhau trong khoảng hợp lệ; không thấy một mức trần số model trong cache.

`invalidate` xóa snapshot và models, ghi lý do. Nó không hủy promise snapshot đang chạy. Một tải cũ có thể hoàn tất sau invalidate và gán lại snapshot trong điều kiện cạnh tranh; đây là khả năng từ hiện thực, cần kiểm chứng bằng test race trước khi đưa ra cam kết “refresh luôn lập tức thay mọi dữ liệu đang tải”. Tương tự, nhiều instance Node có cache độc lập và không tự lan truyền invalidate.

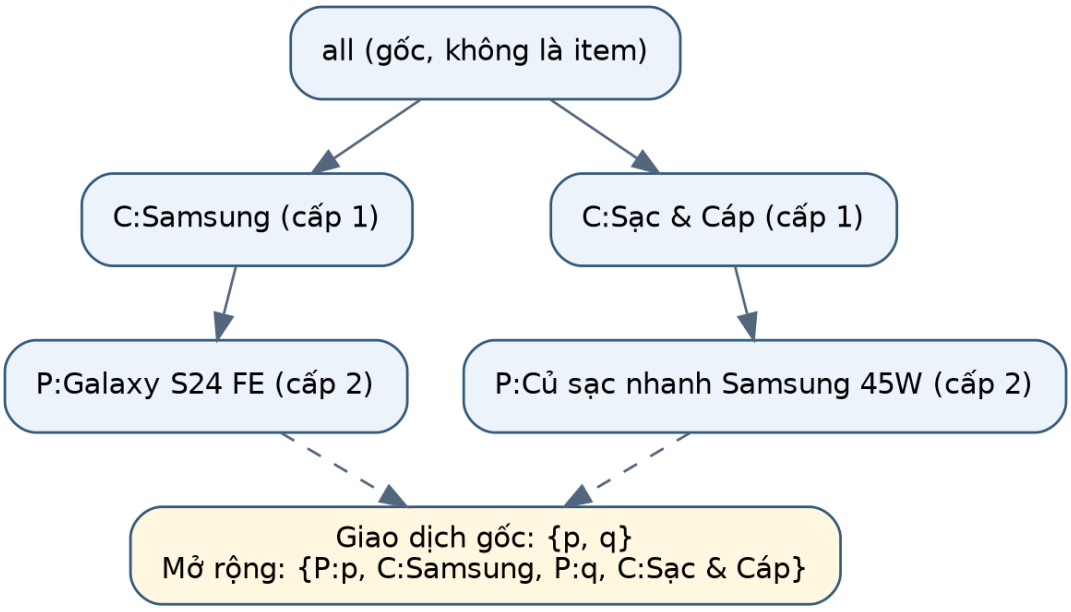
Fallback xây pool còn hàng chưa chọn. Danh sách cùng danh mục được sắp theo số đơn chứa sản phẩm; tiếp theo danh sách bán chạy toàn catalog cũng sắp theo số đơn. Dùng Set tránh đưa lại cùng sản phẩm ở hai lượt. “Bán chạy” ở đây là số đơn có sản phẩm sau loại trùng, không phải số đơn vị bán, doanh thu hoặc lượt nhấp.

5.9. M1 – Khai phá luật kết hợp đa cấp độ

5.9.1. Hierarchy và quan hệ sản phẩm–danh mục

`hierarchyFromCatalog` tạo ID có tiền tố `C:` cho danh mục và `P:` cho sản phẩm. Gốc `all` không là item vì xuất hiện trong mọi giao dịch sẽ tạo các luật hiển nhiên. Hàm kiểm tra product trùng ID, category_id thiếu; `createHierarchy` kiểm tra nút cha tồn tại, ID trùng và chu trình. Những kiểm tra này cần thiết vì một hierarchy sai làm sai support mở rộng hoặc gây duyệt vô hạn.

`extendTransactions` thêm tổ tiên rồi Set để loại trùng. `generalizeTransactions` thay mục bằng tổ tiên tại một cấp cho cách khai phá theo từng mức. Đồ án không tạo tầng danh mục cha không có trong DB. Danh mục điện thoại và phụ kiện dùng trong M2 là phân loại tri thức theo tên, không phải một collection taxonomy ba tầng được thêm vào schema.



Hình 5.1. Nguyên tắc mở rộng giao dịch M1 từ quan hệ category_id.

5.9.2. Support và khai phá theo cấp

`resolveThresholds` hỗ trợ uniform, reduced và group. Reduced yêu cầu ngưỡng cấp sâu không lớn hơn cấp trước; group cấm một item thuộc nhiều nhóm ghi đè và kiểm tra item có trong hierarchy. Ngưỡng mỗi item là ngưỡng nhóm nếu có, nếu không là ngưỡng cấp. Với tập S:

$$s_{\min}(S) = \min_{x \in S} s_{\min}(x), \qquad \theta(S) = \lceil N s_{\min}(S) \rceil. \qquad (5.1)$$

Đặt $s_* = \min_x s_{\min}(x)$. `mineMultilevel` khai phá toàn bộ giao dịch mở rộng tại s_*, bỏ tập tổ tiên–hậu duệ, sau đó giữ tập đạt ngưỡng riêng. Vì ngưỡng riêng không thấp hơn s_*, mọi tập cần giữ đều có trong kết quả khai phá gốc. Để sinh luật, code dùng danh sách nonTrivial còn đầy đủ support tập con, rồi chỉ giữ luật có hợp hai vế đạt ngưỡng riêng. Đây là điểm quyết định tránh sai logic đóng theo tập con khi dùng ngưỡng khác nhau.

Luật được gắn type `level-1`, `level-2` hoặc `mixed`. `mineLevelByLevel` là cách đối chiếu thuần từng cấp với uniform/reduced, có cắt theo tổ tiên phổ biến khi uniform. Nó không trả đầy đủ các luật mixed của `mineMultilevel`; không so hai danh sách mà bỏ qua khác biệt phạm vi.

5.9.3. Loại luật tầm thường và phát hiện dư thừa

`containsAncestorPair` loại mọi itemset có cả mục và tổ tiên, nên luật p → cha(p) và chiều ngược từ cùng cặp không xuất hiện. Đây là loại theo ngữ nghĩa taxonomy. Hàm `findRedundantRules` lại xét một luật cụ thể và luật tổ tiên có trong tập luật, ước lượng support/confidence kỳ vọng bằng tỷ lệ support của các mục được thay.

Với luật tổng quát r_g và luật cụ thể r_s, hiện thực dùng:

$$E[s(r_s)] = s(r_g) \prod_{x \to z} \frac{n(x)}{n(z)}, \quad E[c(r_s)] = c(r_g) \prod_{x \to z \text{ ở vế phải}} \frac{n(x)}{n(z)}. \qquad (5.2)$$

Luật được đánh dấu dư thừa nếu cả support và confidence gần kỳ vọng trong tolerance mặc định 0,1. Code giới hạn `maxAncestorCombos=4096`. Công thức tích nhiều phép

thay là cách hiện thực mở rộng ý tưởng so kỳ vọng; không phải khẳng định các tỷ lệ phản ánh độc lập thống kê thật. Đặc biệt, factory gợi ý M1 không tự gọi `findRedundantRules`; không viết rằng mọi luật dư thừa kiểu kỳ vọng đã bị loại trong danh sách gợi ý.

5.9.4. Ánh xạ luật danh mục về sản phẩm

M1 sử dụng tiền đề là p hoặc danh mục C của p, hậu đề là sản phẩm q hoặc danh mục D. Bốn dạng là p → q, p → D, C → q và C → D. Với hậu đề D, các sản phẩm con được cho trọng số theo bằng chứng đồng xuất hiện:

$$\begin{aligned} w(q \mid X, D) &= \frac{n(X \cup \{q\})}{n(X \cup \{D\})}, \\ score(q) &= conf(X \Rightarrow D)\, w(q \mid X, D). \end{aligned} \tag{5.3}$$

Các số đếm được lấy từ các tập phổ biến đã khai phá, không quét lịch sử tùy ý để bổ sung. Nếu thiếu cặp X,q, `weight` lùi X lên cha; nếu vẫn không có bằng chứng thì w=0. Khi các số đếm dùng cùng X và đều có, tích bằng n(X,q)/n(X), tức confidence của luật cụ thể. Khi lùi ngữ cảnh để ước lượng w nhưng vẫn nhân confidence của luật nguồn ban đầu, không được tự diễn giải kết quả như một xác suất có điều kiện được đo trực tiếp cho riêng sản phẩm hiếm.

Policy `specific-first` ưu tiên số nút sản phẩm trong luật rồi score, lift, support và ID. Policy `score` đặt score trước specificity. Mỗi sản phẩm chỉ giữ ứng viên tốt nhất theo comparator; sau đó lọc khả dụng, áp candidateFilter nếu được ghép M2 và điền fallback nếu bật. Cơ chế lùi cấp giúp tăng cơ hội có bằng chứng, nhưng quan hệ của danh mục không nhất thiết đúng với mọi sản phẩm con.

5.9.5. Ví dụ kiểm thử nhỏ có thể tái hiện

Fixture trong `test/multilevel/recommender-evaluation.test.js` gồm 200 giao dịch: 60 lần {s1,c1}, 50 lần {s2,c2}, 20 lần {s1,ch1}, 20 lần {s2,ch1}, 2 lần {sR,ch1}, 1 lần {sR,cR}, 10 lần {s1,c3} và 37 lần {x1}. Danh mục S chứa s1,s2,sR; K chứa c1,c2,cR,c3; H chứa ch1; c3 hết hàng. Đây là fixture kiểm thử mang tính minh họa của repository, không phải dữ liệu khách hàng.

Với support 0,05, ngưỡng là 10. sR chỉ có 3 đơn nên không tạo luật một cấp. Ở cấp danh mục, S có 163 đơn; c1 có 60 đơn cùng S, c2 có 50, ch1 có 42. M1 ở chế độ rules-only, specific-first trả c1,c2,ch1 cho sR; c3 bị bỏ vì hết hàng, cR thiếu bằng chứng. Ví dụ này chứng minh cơ chế dùng luật danh mục, không chứng minh ba gợi ý đều phù hợp nghiệp vụ cho mọi sản phẩm của S.

Đối với s1, số đơn là 90. Luật $s1 \rightarrow K$ có confidence 70/90; ánh xạ về c1 có w=60/70, cho score=60/90. Kết quả bằng confidence $s1 \rightarrow c1$. Đây là một phép kiểm tra nhất quán của ánh xạ khi có đủ support cụ thể; dữ liệu và phép tính đều có trong fixture test.

5.9.6. Ví dụ từ danh mục TrungShop và giới hạn

Lần chạy minh họa mới dùng R1b, seed 20261101, 20.000 đơn; chỉ lấy train∪validation 16.000 đơn. Ngưỡng sản phẩm 0,002, danh mục 0,005, confidence 0, policy specific-first. Đây là cấu hình minh họa được ghi rõ, không phải tuyên bố cấu hình tối ưu đã chọn trong benchmark cũ. Galaxy S24 FE có 51 đơn lịch sử; luật một cấp ở cấu hình này không có gợi ý đạt điều kiện, nên danh sách mặc định chỉ có fallback.

M1 trả lần lượt: Củ sạc nhanh Samsung 45W (score 0,144024), Ốp lưng Galaxy S24 Ultra (0,124966), Kính cường lực Galaxy S24 Ultra (0,089573), Tai nghe có dây Samsung IA500 (0,087122) và Ốp lưng trong suốt iPhone 14 series (0,024231), đều có nhãn association-rule qua ngữ cảnh phân cấp. Kết quả cho thấy M1 phục hồi được một gợi ý sạc nhưng đồng thời mang theo gợi ý ốp/kính khác dòng máy. Không thể kết luận "đa cấp luôn cải thiện chất lượng" chỉ vì có nhiều luật hơn.

Ưu điểm của M1 là tận dụng quan hệ nhóm khi mục cụ thể ít giao dịch và giữ được nguồn luật. Hạn chế là giao dịch mở rộng dài hơn, số tập tăng, ngưỡng nhiều hơn, quan hệ tổng quát có thể thiếu phù hợp ở cấp sản phẩm và không tự có tri thức phần cứng. M1 chưa được tích hợp vào website; các kết quả này chỉ là thực thi offline.

5.10. M2 – Khai phá dựa trên ràng buộc

5.10.1. Ràng buộc nghiệp vụ và dữ liệu sử dụng

`buildBusinessRules` phân loại phone theo tên category trong danh sách iPhone, Samsung, Xiaomi, OPPO, Vivo, Nokia; các category khác được coi là accessory. Cần truyền thêm `category` là tên danh mục trong catalog nghiên cứu. Chỉ có category_id mà thiếu tên sẽ không đủ để cách phân loại này nhận diện đúng điện thoại; đây là điều kiện cần nếu sau này tích hợp vào API đang dùng loader sản phẩm khác định dạng.

`modelOf` nhận diện ốp lưng/kính cường lực ở đầu tên, bỏ một số từ silicon/trong suốt và đuôi series, rồi chuyển về chữ thường. Phụ kiện không khớp mẫu được coi là dùng chung. `fits` dùng phép includes giữa tên điện thoại và tên dòng máy đã rút ra. Đây là heuristic văn bản: "sạc, cáp, tai nghe, pin dùng chung" là giả định của mô hình, không phải chứng nhận tương thích công suất, cổng kết nối, giao thức hay kích thước.

Bảng 5.3. Ràng buộc M2-A và M2-B trong `business.js`.

Mã	Ràng buộc	Nơi áp dụng
A1	Một sản phẩm → một sản phẩm	Chọn luật/ứng viên
A2	Nếu tiền đề là điện thoại thì hậu đề là phụ kiện	Lọc cặp có hướng
A3	Cặp phù hợp heuristic tên dòng máy	Lọc luật và fallback
A4	Sản phẩm gợi ý còn hàng	Chỉ hậu đề
B1	count(S) ≤ 2	Cắt tỉa tập mục
B2	count(S ∩ điện thoại) ≤ 1	Cắt tỉa tập mục
B3	Không có cặp không tương thích	Cắt tỉa tập mục

B2 cần được hiểu trong mục tiêu nghiên cứu mua kèm phụ kiện: cặp hai điện thoại bị bỏ, dù chúng có thể cùng nằm trong một đơn hợp lệ. M2 không phải luật xác thực đơn hàng và không từ chối khách mua hai điện thoại; nó giới hạn không gian mẫu/ứng viên gợi ý theo mục tiêu đã chọn.

5.10.2. Push và post trong Apriori, FP-Growth

`constrainedApriori` dùng chung các khối `findFrequent1Itemsets` và `aprioriGen`, lọc ứng viên vi phạm trước đếm. Vì vi phạm chống đơn điệu, việc bỏ ứng viên cũng ngăn sinh mọi siêu tập từ nó ở mức sau mà không mất mẫu thỏa. Các số generated, prunedByConstraint và counted được ghi riêng để giải thích chi phí.

`constrainedFPGrowth` kiểm tra mẫu beta trước xuất và trước dựng cây điều kiện; trong conditional pattern base, mục x bị bỏ nếu beta∪{x} vi phạm. Single-path shortcut cũng kiểm tra ràng buộc cho từng mở rộng. Tính an toàn dựa vào ràng buộc chống đơn điệu, không phải vì mọi điều kiện kinh doanh đều có thể đẩy vào cây.

`assertPushable` từ chối constraint không có tính chất phù hợp. `postFilter` là lọc sau khai phá gốc. Với cùng các ràng buộc chống đơn điệu, hai chế độ phải giữ cùng các itemset và support, nhưng thời gian có thể khác. Ràng buộc hậu lọc không chống đơn điệu có thể làm danh sách không còn đóng theo tập con; không truyền tùy tiện kết quả đó vào `generateRules` mà thiếu support cần thiết.

Đối với tồn kho A4, code chỉ lọc hậu đề và fallback, không đẩy thành “mọi mục còn hàng”. Nếu p hết hàng nhưng q còn hàng, hệ thống vẫn có thể gợi ý q. Đây là ví dụ cụ thể vì sao phải phân biệt điều kiện của một tập vô hướng với điều kiện của hai vế luật có hướng.

5.10.3. Ví dụ xử lý và kết quả

Ví dụ nghiệp vụ dựa trên tên trong catalog: tiền đề là Samsung Galaxy S24 FE 128GB. Một ứng viên Samsung Galaxy S24 Ultra là điện thoại nên vi phạm A2. Ốp lưng Galaxy S24 Ultra vi phạm A3 vì tên dòng máy không nằm trong tên điện thoại đang xem. Củ sạc nhanh Samsung 45W không có model-specific nên qua heuristic A3 nếu còn hàng; điều này không chứng minh sạc được kiểm nghiệm kỹ thuật trên máy.

Trong lần chạy R1b minh họa mục 5.9, M2 một cấp với support 0,002/confidence 0 không tạo gợi ý từ luật cho Galaxy S24 FE. Sau lọc fallback, danh sách là Củ sạc nhanh USB-C 20W, Pin dự phòng Xiaomi 10.000mAh 20W, Tai nghe Apple AirPods 2, Pin dự phòng Samsung 20.000mAh 45W và Cáp Apple USB-C to USB-C 1m. Cả năm đều

mang source fallback. Việc danh sách không chứa điện thoại không đồng nghĩa có năm luật mới; đây là tác động của tri thức lọc và thứ tự phổ biến.

Với Apriori, ứng viên {Galaxy S24 FE, Galaxy S24 Ultra} có hai phone nên B2 loại trước đếm; tập {Galaxy S24 FE, Ốp Galaxy S24 Ultra} bị B3 loại. Với post, các ứng viên này vẫn có thể được đếm rồi mới bỏ. Các cặp thỏa còn lại phải có support giống nhau giữa hai cách. Chương 6 trình bày kết quả benchmark lưu trong repository, gồm cả trường hợp push FP-Growth chậm hơn post.

5.10.4. Ưu điểm và hạn chế

M2 giúp kiểm soát danh sách theo mục tiêu phụ kiện, giảm mẫu không cần thiết và cung cấp lý do vi phạm. Có thể ghép M1 thông qua constraintsFor và candidate-Filter/fallbackFilter. Nhưng tính đúng của tên category và tên model là tiền điều kiện; các từ viết tắt, tên thiếu, khác cách đặt hoặc dòng máy có tên chứa nhau có thể gây bỏ sót/nhận nhầm.

Tỷ lệ hợp lý nghiệp vụ 100% trên các bảng nghiên cứu có nghĩa mọi mục thỏa chính bộ kiểm tra A2–A4. Nó không là độ chính xác được chuyên gia phần cứng độc lập chấm, không là mức hài lòng khách hàng và không xác nhận tính đầy đủ của taxonomy. M2-B có thể giảm công việc khai phá nhưng có overhead kiểm ràng buộc; ít mẫu hơn không bảo đảm luôn chạy nhanh hơn.

5.11. M3 – Xếp hạng và đa dạng hóa kết quả

5.11.1. Nguồn ứng viên và chiến lược

`fitBase` hỗ trợ flat, m2-flat, m1 và m2-m1. `flatCandidateFn` tạo ứng viên từ luật một cấp; `multilevelCandidateFn` nhận ứng viên từ M1 và xây các đại lượng cần tính độ đo. `buildRanked` dùng cùng nguồn ứng viên nền, đổi thứ tự rồi chọn Top-K. Chiến lược native giữ thứ tự gốc và được kiểm tra đối chiếu; khác biệt chất lượng khi dùng chiến lược khác cần quy cho thứ tự/chọn danh sách, không cho một lần khai phá dữ liệu mới.

Các đại lượng của ứng viên là joint, supX, supQ và N. Với luật cụ thể joint là số đếm thật. Với ứng viên phân cấp có ánh xạ, joint được ước lượng bằng score M1 × supX. Khi

đó Kulc/IR tính từ ứng viên phân cấp có thể dùng đại lượng ước lượng, không mặc định xem là bảng đếm quan sát trực tiếp của cặp p,q.

Bảng 5.4. Chiến lược `scoreOf` và `rankCandidates`.

Strategy	Điểm	Vai trò tham số
native	Giữ thứ tự nền	Không tạo một thang điểm mới
confidence	joint/supX	Ưu tiên xác suất có điều kiện
kulc-ir	Kulc × (1 − β IR)	β tăng mức phạt mất cân bằng
popularity	confidence × popularity^(−γ)	γ tăng ưu tiên tương đối cho mục ít phổ biến

Nếu hòa điểm trong dung sai, comparator dùng lift, jointCount, nativeRank rồi ID. Giữ thứ tự ổn định quan trọng cho tái lập: một thay đổi trong thứ tự hòa có thể làm thay mục thứ 5 và ảnh hưởng hit/coverage dù điểm chính bằng nhau.

5.11.2. Công thức và nguồn gốc

$$score_{KI}(X, q) = Kulc(X, q)\,[1 - \beta IR(X, q)]. \qquad (5.4)$$

$$pop(q) = \frac{n(q)}{N}, \qquad score_{pop}(X, q) = conf(X \Rightarrow q)\, pop(q)^{-\gamma}. \qquad (5.5)$$

Kulc và IR là độ đo nền trong mục 6.3.3 của sách. Phép nhân phạt ở (5.4) là lựa chọn của đồ án; beta được chọn trên validation, không là tham số do giáo trình quy định. Công thức (5.5) nội suy confidence–lift ở gamma từ 0 đến 1; ở gamma=1 chính là lift. Dữ liệu cho ứng viên hợp lệ cần pop>0 để tránh lũy thừa âm tại 0.

Với ví dụ N=10, n(X)=6, n(q)=5, joint=4 ở Chương 2, $\beta = 0,5$ cho $score_{KI} \approx 0,733333 \times (1 - 0,5/7) = 0,680952$. Với $\gamma = 0,5$, $score_{pop} \approx 0,942809$; với $\gamma = 1$, điểm bằng $1,333333$, đúng lift. Đây là ví dụ toán học minh họa công thức, không phải số đo thực nghiệm trên đơn TrungShop.

5.11.3. Đa dạng hóa theo danh mục

Với m ứng viên đã xếp, code dùng thứ hạng r từ 0. Ở mỗi lượt chọn:

$$i^* = \arg\max_{i \notin S} \left\{ \lambda \left(1 - \frac{r_i}{m}\right) - (1-\lambda)\mathbf{1}[c(i) \in c(S)] \right\}. \qquad (5.6)$$

S là danh sách đã chọn, $c(i)$ là danh mục, lambda trong [0,1]. Lambda=1 giữ nguyên thứ tự; lambda nhỏ hơn khuyến khích danh mục mới. Hòa được giải bằng thứ tự ban đầu. Mức phạt chỉ là nhị phân "đã có hay chưa", không tăng theo số lần lặp danh mục và không đo tương đồng nội dung giữa các sản phẩm.

Ví dụ sư phạm với thứ tự A1,A2,B1 và m=3, A1/A2 cùng danh mục A, B1 thuộc B, lambda=0,5: chọn A1 trước; ở lượt sau A2 có giá trị 0,5×(2/3)−0,5=−1/6, B1 có 0,5×(1/3)=1/6, nên chọn B1 rồi A2. Kết quả A1,B1,A2 là tác động của phạt danh mục, dù A2 vốn xếp trên B1.

Trong `buildRanked`, luật luôn được chọn trước fallback. Nếu còn chỗ, fallback cũng được đa dạng hóa và biết danh mục đã có ở phần luật. Vì vậy mô tả M3 chỉ đa dạng hóa các luật là thiếu; phần fallback có thể thay đổi đáng kể và tạo hiệu quả trên truy vấn không có luật nào.

5.11.4. Ví dụ chạy trên dữ liệu TrungShop

Với lịch sử R1b minh họa, iPhone 15 128GB có 1.146 đơn. Luật một cấp xếp Ốp lưng silicon iPhone 15, Kính cường lực iPhone 15, Củ sạc nhanh USB-C 20W, Cáp Apple USB-C to USB-C 1m và Tai nghe Apple AirPods 2 vào Top-5. Các confidence tương ứng xấp xỉ 0,415358; 0,286213; 0,226876; 0,183246; 0,096859.

Chạy M3 nền flat với kulc-ir, beta=0,5 và lambda=0,75 cho thứ tự cuối: Ốp lưng silicon iPhone 15 (điểm 0,371460), Cáp Apple USB-C to USB-C 1m (0,184465), Kính cường lực iPhone 15 (0,307393), Tai nghe Apple AirPods 2 (0,090988), Củ sạc nhanh USB-C 20W (0,171247). Điểm không giảm đơn điệu trong danh sách cuối vì đã có bước đa dạng hóa dựa trên thứ hạng và danh mục. Toàn bộ năm mục vẫn có source association-rule.

Với Galaxy S24 FE không có ứng viên từ luật ở cấu hình này, M3 nền flat thay đổi thứ

tự fallback và vẫn có thể chọn ốp iPhone vì không ghép M2. Chọn được nhiều danh mục hơn không bảo đảm tương thích. Các ví dụ lưu đầy đủ ID, luật nguồn, điểm và source trong tệp bằng chứng, giúp phân biệt tác động ranking với tác động của bộ lọc.

5.11.5. Ưu điểm, hạn chế và vị trí học thuật

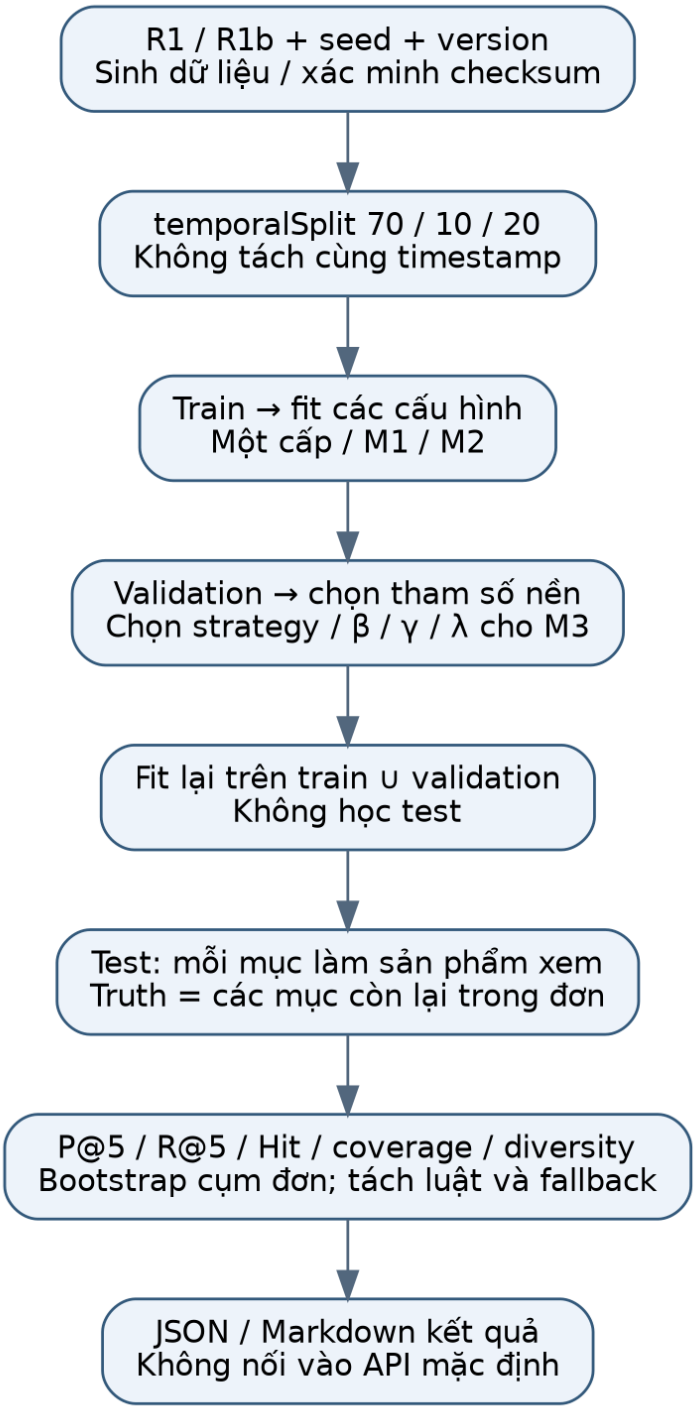
M3 không cần khai phá lại cho từng chiến lược xếp hạng trên cùng mô hình nền và giữ được bằng chứng ứng viên. Đa dạng hóa có thể tránh danh sách quá lặp danh mục. Tuy nhiên, giới hạn nguồn ứng viên vẫn giữ nguyên: một sản phẩm bị miner hoặc M2 loại không được điểm M3 tự phục hồi. Xếp hạng có thể làm giảm catalog coverage dù độ đa dạng từng danh sách tăng.

Phương pháp tham lam là chuyển thể theo tinh thần marginal relevance và redundancy-aware top-k, không phải thuật toán nguyên bản của mục 7.5.2 áp dụng nguyên vẹn. Mức tương đồng chỉ là cùng category, không là một hàm khoảng cách semantic. Lựa chọn hai giai đoạn (tham số nền trước, xếp hạng sau) có thể bỏ lỡ tổ hợp tốt hơn nếu tối ưu chung.

5.12. Quy trình gợi ý tổng thể

Luồng đã tích hợp là orders → transactions → frequent itemsets → rules → single-item index → ranking/lọc → fallback → gắn giá → UI. M1 thay cách biểu diễn và khai phá/ánh xạ trong nghiên cứu; M2 thêm điều kiện mẫu và cặp ứng viên; M3 thay thứ tự/chọn Top-K. Các factory nghiên cứu có thể phối hợp nhưng không vì dùng lại service helper mà trở thành một phần API đang hoạt động.

Bước chọn tham số chỉ học train và chấm validation. Sau chọn, mô hình được fit lại trên train∪validation rồi chấm test. Popularity và fallback phải được tính từ cùng lịch sử được phép; không lấy itemCounts toàn bộ orders của website khi chấm một phần test offline. Nếu dùng cache online không đúng thời điểm sẽ tạo rò rỉ dữ liệu ngay cả khi thuật toán mining không đổi.



Hình 5.2. Quy trình thực nghiệm M1–M3 và ranh giới train/validation/test.

5.13. Phân tích độ phức tạp

Gọi N là số giao dịch, m là số mục phân biệt, L là độ dài giao dịch lớn nhất, F là số tập phổ biến và R là số luật. Chuẩn hóa cần đọc tổng số mục và sắp từng giao dịch, có thể chặn trên bằng $O(NL \log L)$. ItemCounts có chi phí tuyến tính theo tổng độ dài sau chuẩn hóa. Đọc snapshot và giữ nó trong tiến trình cần bộ nhớ tỷ lệ với tổng số mục đã đọc.

Apriori có số ứng viên mức k tới $\binom{m}{k}$ trong trường hợp xấu; hiện thực đếm bằng lựa chọn giữa liệt kê tập con giao dịch và duyệt ứng viên. Một ước lượng phần đếm ở mức k là tổng theo giao dịch của giá trị nhỏ hơn giữa công việc liệt kê $\binom{|T|}{k}$ và kiểm tra $|C_k|$ ứng viên, có thêm chi phí dựng khóa/kiểm từng mục. Không đủ chính xác nếu chỉ ghi chung Apriori là O(N) hoặc O(m²).

FP-tree ban đầu có số nút không vượt tổng số mục phổ biến trong các giao dịch, nhưng tổng công việc dựng cây điều kiện phụ thuộc cấu trúc chia sẻ tiền tố. Trong trường hợp nhiều mẫu phổ biến, đầu ra tự nó đã có thể là $2^m - 1$ tập. FP-Growth không có bảo đảm đa thức tổng quát chỉ vì không sinh candidate như Apriori.

Sinh luật từ các tập kích thước l cần $2^l - 2$ cách chia mỗi tập, nên tổng công việc tỷ lệ với tổng các cách chia và chi phí tạo khóa/độ đo. M1 làm giao dịch dài thêm các tổ tiên và có thể tăng F; M2 giảm không gian theo ràng buộc nhưng thêm chi phí kiểm tra; M3 sắp m_c ứng viên $O(m_c \log m_c)$ và chọn tham lam Top-K khoảng $O(K m_c)$, cộng chi phí xử lý mảng. Đây là phân tích cấu trúc mã, không thay cho benchmark hoặc chặn trên tài nguyên production.

5.14. Ví dụ chạy xuyên suốt từ đơn hàng đến gợi ý

Ví dụ thực thi mới lấy dữ liệu giả lập R1b/20261101, kiểm tra checksum trùng manifest rồi tách thời gian. Chỉ 16.000 đơn lịch sử được dùng cho mining. Các ObjectId trong catalog là ID xác định từ bộ sinh, không lấy danh tính khách hàng. Với iPhone 15, support count singleton là 1.146; số đơn cùng Ốp lưng silicon iPhone 15 là 476, nên confidence bằng 476/1146≈0,415358.

MinSupport=0,002 tương ứng 32 đơn; cặp trên đạt ngưỡng. `generateRules` tạo luật có

tiền đề iPhone 15 và hậu đề ốp; minConfidence=0 trong cấu hình minh họa không loại luật này. `indexSingleItemRules` đưa nó vào chỉ mục của iPhone 15. `rankRecommendations` kiểm lift>1 và sản phẩm còn hàng, chọn ốp ở vị trí đầu. Các mục có luật đủ năm nên không thêm fallback ở truy vấn này.

Nếu dùng ngưỡng mặc định website 0,01 và confidence 0,2 thì mô hình và danh sách có thể khác; ví dụ minh họa không được gọi là đầu ra mặc định production. Phần thực nghiệm so sánh cấu hình mặc định và cấu hình chọn validation bằng hai dòng riêng. Ví dụ mới chỉ giải thích cơ chế, không được cộng vào bộ 5 seed như một thực nghiệm mới có hiệu năng đo tương đương.

Đầu ra API có thể giữ product, rule, explanation và metadata cache. UI ở chế độ khách hiển thị thẻ sản phẩm theo nguồn, không đưa mọi chỉ số kỹ thuật vào hành trình mua hàng. Chế độ research có thể hiện confidence/support/lift để kiểm chứng. Kết luận có cơ sở là gợi ý có thể truy vết đến lịch sử và điều kiện lọc; chưa có cơ sở kết luận xác suất mua tương lai của một khách cụ thể bằng đúng confidence lịch sử.

CHƯƠNG 6. THỰC NGHIỆM VÀ ĐÁNH GIÁ

6.1. Môi trường thực nghiệm

Báo cáo phân biệt hai loại bằng chứng. Loại thứ nhất là các kết quả đã lưu trong `research-results`, được tạo ngày 26–27/10/2026, với metadata Node v20.20.2, win32 x64 và CPU Intel Core i7-1255U. Loại thứ hai là kiểm tra mới trong môi trường Linux của phiên khảo sát ngày 02/11/2026, gồm kiểm thử, tái sinh dữ liệu và ví dụ đầu ra. Không chuyển thời gian đo của máy cũ thành thành tích đo trên máy mới.

Benchmark R1/R1b khai phá train∪validation gồm 16.000 đơn. Mỗi lượt dùng một tiến trình con, có giới hạn heap 2.048 MB và thời gian 300 giây; số đo thời gian là trung vị 15 lần sau một lần chạy lạnh. RSS trong bảng là mức tăng bộ nhớ đỉnh lần chạy lạnh, dữ liệu được sinh ở tiến trình cha nên không bao gồm toàn bộ chi phí tạo dataset. Các giá trị là trung bình qua 5 seed của thống kê từng seed.

Benchmark mining giai đoạn đầu có cách đo khác: trung vị 7 lần, các bộ synthetic thay đổi độ dày, số đơn và ngưỡng. Không gộp các lần đo này với benchmark R1/R1b thành một bảng thời gian đồng nhất mà bỏ qua phương pháp. File JSON và Markdown được giữ trong gói bằng chứng để tra lại cấu hình và giá trị gốc.

6.2. Dữ liệu thực nghiệm

Bảng 6.1. Phân biệt dữ liệu demo, dữ liệu nghiên cứu và dữ liệu benchmark.

Bộ dữ liệu	Quy mô	Đặc điểm	Phạm vi diễn giải
GR2-demo tái tạo offline	1.000 đơn, 41 sản phẩm	Seed 20261026, luật mua kèm cài sẵn	Kiểm tra pipeline và baseline quy mô nhỏ
R1	5 seed × 20.000 đơn, 41 sản phẩm	Giống bộ sinh demo; 55% nhánh điện thoại, 35% phụ kiện, 10% nhiễu	Đo khả năng phục hồi cấu trúc giả lập
R1b	5 seed × 20.000 đơn, 48 sản phẩm	Nhiễu 25%, 7 sản phẩm hiếm; tỷ lệ mua kèm giảm	Khảo sát thưa dữ liệu và độ bền
Synthetic mining	1.000–50.000 giao dịch tùy cấu hình	Thay ngưỡng/độ dài/tập mục	Benchmark thuật toán, không phải đánh giá khách hàng

Các seed nghiên cứu là 20261101, 20261102, 20261103, 20261104 và 20261105. Chúng

là số nguyên điều khiển PRNG, không phải ngày xảy ra thực nghiệm; không suy luận có thử nghiệm trong tương lai từ hình thức chuỗi seed. Khoảng sinh đơn nghiên cứu được cấu hình từ 01/04/2026 đến 25/10/2026 theo UTC+7.

Manifest lưu `generatorVersion`, cấu hình, checksum dữ liệu và thống kê. Ví dụ R1 seed20261101 có checksum `e84322bb341aeb1d1ba300c07f06479f79de67dc`. GR2-demo có checksum đơn hàng `d1eb602b9280f67944583f56b7e9f832539a7925`. Checksum giúp phát hiện thay đổi nguồn, không phải bảo đảm dữ liệu mô phỏng đại diện cho thị trường thật.

Không tìm thấy bằng chứng về tập đơn mua thực của khách hàng đã được cung cấp để đánh giá. Dữ liệu demo được tái sinh trong bộ nhớ và không chứng minh MongoDB hiện đang chứa đúng 1.000 đơn. [CẦN BỔ SUNG: dữ liệu giao dịch thật đã ẩn danh, khoảng thời gian, quy tắc lọc trạng thái và quyền sử dụng.]

6.3. Cách sinh dữ liệu đơn hàng

`scripts/seed-demo/orders.js` chứa trọng số điện thoại PHONE_WEIGHTS, quan hệ COMPANIONS và gói ACCESSORY_BUNDLES. `research/datasets/generator.js` tái sử dụng các bảng và PRNG để tạo R1. R1b đổi tỷ lệ các nhánh: 43,8% điện thoại, 30% phụ kiện, 1,2% sản phẩm hiếm và 25% nhiễu. Xác suất mua kèm của nhánh thông thường được nhân 0,7; nhánh hiếm có các quan hệ riêng được khai báo.

Các tỷ lệ là xác suất chọn nhánh, không bảo đảm mỗi bộ 20.000 đơn có đúng tuyệt đối số lượng tương ứng. Các bảng quan hệ được lập trước khiến dữ liệu có mẫu để miner tìm lại. Chẳng hạn Galaxy S24 FE gắn với Ốp Galaxy S24 FE xác suất 0,7 và sạc Samsung 45W xác suất 0,4 trong nhánh hiếm. Một ràng buộc tên chọn đúng ốp có thể khớp với quy luật bộ sinh mà không cần khai phá được luật hiếm đó.

Sau sinh, các sản phẩm có ID xác định từ key, đơn có thời gian và items loại trùng. Dataset sắp theo thời gian và băm cả cấu hình, catalog, orders để có checksum. Các script đánh giá nghiên cứu không kết nối MongoDB. Việc chạy lại sinh dataset trong bộ nhớ là thao tác đọc/tính, khác với chạy `seed-demo.js` ghi vào DB; phiên khảo sát không seed hay sửa dữ liệu cửa hàng.

6.4. Thiết lập tham số và chống rò rỉ

`temporalSplit` dùng tỷ lệ 70/10/20, sắp time rồi id và dời ranh giới để không tách các đơn cùng timestamp. Vì có điều chỉnh ranh giới, không áp một con số chia cứng cho mọi dataset tùy ý nếu chưa kiểm tra. Với các bộ nghiên cứu đã lưu, lịch sử train∪validation là 16.000 đơn. Học tham số từ train, chấm validation, sau đó học lại train∪validation và chấm test.

`buildQueries` lấy mỗi sản phẩm trong một đơn có ít nhất hai sản phẩm khác nhau làm sản phẩm đang xem; các sản phẩm còn lại là truth. Giao thức giữ lại một mục làm ngữ cảnh, không phải truyền toàn bộ phần giỏ còn lại cho recommender. Một đơn sinh nhiều truy vấn có tương quan, nên bootstrap theo cụm đơn thay vì coi chúng hoàn toàn độc lập.

Các mặc định website là support 0,01, confidence 0,2, K=5. Baseline chọn validation dùng lưới ngưỡng khác và không được gọi là cùng cấu hình mặc định. M1 chọn ngưỡng danh mục/sản phẩm và policy; M2 chọn các ngưỡng nền; M3 chọn strategy, beta/gamma và lambda sau tham số nền. M3-R tối ưu recall validation; M3-C tối ưu coverage trong các cấu hình recall validation không thấp hơn mức tốt nhất quá 0,5 điểm phần trăm. Đây là tiêu chí đã khai báo trong code/báo cáo nghiên cứu, không chọn bằng kết quả test.

`assertMiningBudget` ước lượng số singleton phổ biến, số cặp, chiều dài giao dịch và cận trên tổng tập con. Các giới hạn mặc định là 5.000 mục phổ biến, 5.000.000 cặp, độ dài 24 và cận trên 20.000.000. Đây là hàng rào thận trọng trong nghiên cứu; luồng API mining mặc định không gọi trực tiếp bộ kiểm tra ngân sách này.

6.5. Kịch bản kiểm thử

Bảng 6.2. Kịch bản được đối chiếu với bộ test hiện có.

Nhóm	Kịch bản và kết quả mong đợi	Cơ sở kiểm tra
Chuẩn hóa	Mục trùng chỉ tính một đơn; rỗng giữ mẫu số	`test/mining/edge-cases.test.js`
Apriori	Bảng 6.1 của sách cho L1–L3 đúng	`test/mining/apriori.test.js`
FP-Growth	Header, cơ sở điều kiện, đầu ra đúng	`test/mining/fpgrowth.test.js`
Tương đương	Cùng data/ngưỡng cho cùng itemsets	`test/mining/equivalence.test.js`

Nhóm	Kịch bản và kết quả mong đợi	Cơ sở kiểm tra
Luật	Confidence/lift/Kulc/IR, thiếu subset phải báo lỗi	`test/mining/rules.test.js`
Gợi ý	Bỏ chính p, trùng, thiếu, hết hàng, lift≤1	Các test recommendation
Cache	Dùng lại model, TTL/fingerprint/invalidate	`test/recommendation/service.test.js`
M1	Mở rộng, ngưỡng riêng, tổ tiên, ánh xạ	Các test multilevel
M2	Push=post, không ràng buộc=miner gốc	Các test constraints
M3	Native=nền; nguồn luật/fallback đúng	`test/ranking/ranking.test.js`
Đánh giá	Không trùng tập, không fit dữ liệu tương lai	Các test evaluation
API nghiệp vụ	Quyền truy cập, giá DB, trạng thái, coupon	Các test security/admin/catalog/coupon

Bảng nêu phạm vi và kỳ vọng, không tự gắn "đạt" cho toàn bộ giao diện. Kết quả chạy mới được ghi trong phần nhật ký kiểm chứng và phụ lục. Một test dùng fake-app/model giả kiểm chứng hợp đồng/controller trong môi trường mô phỏng, không tự chứng minh query chạy trên MongoDB thật hoặc CSS hiển thị đúng trên mọi trình duyệt.

6.6. Kiểm thử chức năng website

Repository có các test frontend cho Cart, CheckoutForm, Account, PriceTag, ProductBrowser, Header và FrequentlyBoughtTogether; phía Admin có test Product, Category, Order, Coupon, User, Comment và Dashboard. Sự hiện diện của test chỉ chứng minh có kịch bản được viết. Phiên khảo sát ưu tiên chạy lại backend/thuật toán và chưa có bằng chứng chạy toàn bộ test React hoặc kịch bản end-to-end trên trình duyệt.

Bộ fake-app phía backend cho phép kiểm tra phân quyền, không lộ password, bảo vệ đơn cá nhân, dùng giá DB, từ chối hàng hết, giới hạn coupon và đổi trạng thái. Những test này hữu ích vì phát hiện sai nghiệp vụ mà không cần dữ liệu thật. Tuy nhiên thiếu bằng chứng trực quan của bản chạy, cấu hình CORS production, phiên làm việc thực và tích hợp địa chỉ ngoài hệ thống.

Test MongoDB GR2 có điều kiện `RECO_DB_IT=1`, mặc định skip; nó chỉ đọc và kiểm tra số bản ghi trước/sau. Báo cáo không đổi skip thành pass hoặc kích hoạt kết nối đến DB

chưa có cấu hình. [CẦN BỔ SUNG: chạy kiểm thử MongoDB thật với dữ liệu phù hợp và lưu log, chạy test Shop/Admin, chụp ảnh giao diện và kiểm thử đặt hàng qua trình duyệt.]

6.7. Kiểm thử thuật toán khai phá

Các fixture tái hiện ví dụ sách giúp kiểm tra support và cấu trúc cây có kết quả biết trước. Test tương đương giữa Apriori và FP-Growth tăng khả năng phát hiện lỗi hiện thực, nhưng hai thuật toán cùng một lỗi chuẩn hóa vẫn có thể cho cùng kết quả sai. Vì vậy cần cả fixture tay, công thức độ đo và kiểm tra các trường hợp ngoại lệ.

Các test edge bao gồm dữ liệu rỗng, mục trùng, ngưỡng không hợp lệ và trường hợp không có tập phổ biến. Test M1 kiểm hệ phân cấp và không xuất luật tổ tiên; test M2 so push/post và tính chất ràng buộc; test M3 so native và nguồn fallback. Tính đúng trên những bộ này là bằng chứng cài đặt, không thay cho đánh giá mức hữu ích với khách hàng.

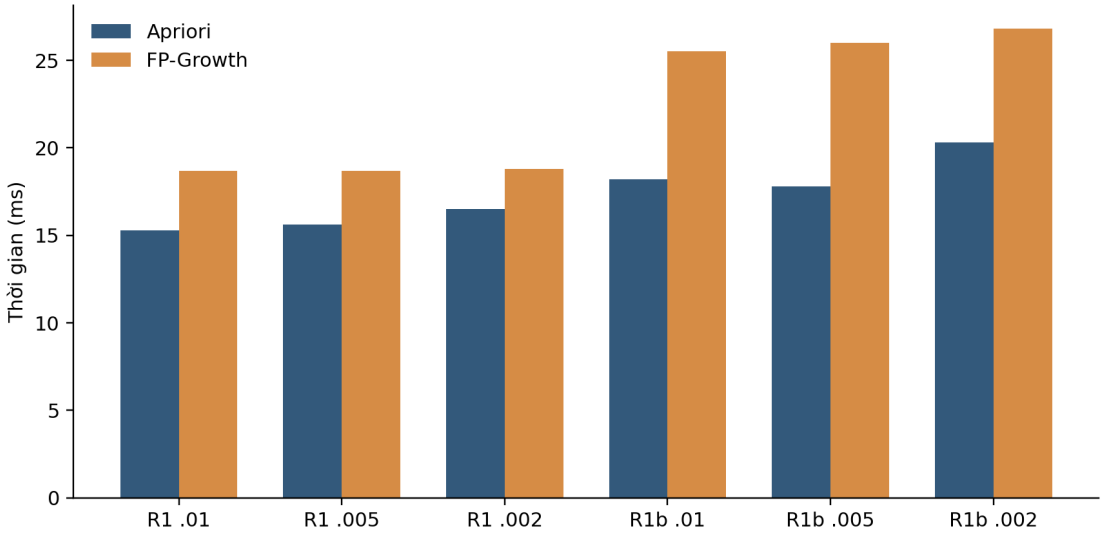
Trong lần khảo sát, tái sinh R1b seed20261101 và chạy ví dụ $p \rightarrow q$ đã thực hiện thành công với Node v24.19.0. Tuy nhiên chạy toàn bộ backend trên Node 24 gặp lỗi tương thích `Utils.isRegExp is not a function` trong dependency `config`. Lỗi môi trường này được giữ trong log ban đầu và không trình bày như lỗi công thức Apriori/FP-Growth. Kết quả kiểm thử bằng runtime phù hợp được ghi riêng trong nhật ký kiểm chứng đi kèm.

6.8. So sánh Apriori và FP-Growth

Bảng 6.3 lấy đúng các dòng trong `phase4-step2-benchmark.md` và đối chiếu artifact JSON. Đơn vị thời gian là ms, TB±SD qua năm seed của trung vị mỗi seed; khai phá 16.000 đơn lịch sử, không gồm API/cache.

Bảng 6.3. Benchmark một cấp trên R1/R1b đã lưu trong repository.

Hồ sơ	Support	Apriori (ms)	FP-Growth (ms)	A/FP	Kết quả
R1	0,01	15,3 ± 0,5	18,7 ± 0,3	0,82	Trùng 5/5
R1	0,005	15,6 ± 0,2	18,7 ± 0,3	0,84	Trùng 5/5
R1	0,002	16,5 ± 0,5	18,8 ± 0,6	0,88	Trùng 5/5
R1b	0,01	18,2 ± 3,9	25,5 ± 3,8	0,71	Trùng 5/5
R1b	0,005	17,8 ± 3,4	26,0 ± 6,3	0,70	Trùng 5/5
R1b	0,002	20,3 ± 2,2	26,8 ± 1,9	0,76	Trùng 5/5



Hình 6.1. Thời gian một cấp R1/R1b theo Bảng 6.3; biểu đồ tái vẽ từ số liệu đã lưu.

Apriori nhanh hơn trong những cấu hình thưa này. Dữ liệu ít mục mỗi đơn làm số ứng viên cần đếm tương đối nhỏ, trong khi dựng cây và cơ sở điều kiện có overhead. Không diễn giải mặc định FP-Growth của website như một kết luận rằng thuật toán đó luôn nhanh nhất. Độ đúng tập phổ biến giống nhau không kéo theo thời gian hoặc bộ nhớ giống nhau.

Bộ synthetic dày trong `benchmark-mining.md` có N=5.000, 100 item, độ dài giỏ trung bình 10. Tại support 2%, Apriori 196,3 ms, FP-Growth 60,7 ms, tỷ lệ khoảng 3,2. Đây là bộ khác R1/R1b, cho thấy lợi thế FP-Growth khi cấu trúc dày hơn và có khả năng nén. So sánh này hỗ trợ giải thích phụ thuộc dữ liệu, không phải phép nội suy độ trễ cho website thật.

6.9. Đánh giá M1

Bảng 6.4 sử dụng các dòng tương ứng từ báo cáo M3 tổng hợp, vì cùng một giao thức cung cấp baseline đã chọn ngưỡng và các mô hình nền. Tất cả số là phần trăm TB±SD qua năm seed test. “Website” là cấu hình mặc định, “một cấp” là cấu hình chọn bằng validation.

Bảng 6.4. So sánh M1 với mặc định và baseline đã chọn ngưỡng.

Hồ sơ/phương pháp	P@5	R@5	Hit@5	Coverage
R1 – Website	22,6 ± 0,4	64,3 ± 0,8	70,3 ± 0,9	97,6 ± 0,0
R1 – Một cấp	28,5 ± 0,4	81,8 ± 0,3	84,1 ± 0,4	89,8 ± 1,1

Hồ sơ/phương pháp	P@5	R@5	Hit@5	Coverage
R1 – M1	28,5 ± 0,4	82,0 ± 0,5	84,3 ± 0,5	83,4 ± 1,1
R1b – Website	11,7 ± 0,3	36,8 ± 0,9	47,4 ± 0,7	89,6 ± 0,0
R1b – Một cấp	17,2 ± 0,4	55,9 ± 1,0	63,1 ± 1,0	86,3 ± 2,4
R1b – M1	17,4 ± 0,3	56,3 ± 0,9	63,8 ± 0,8	81,3 ± 4,2

Lợi ích toàn bộ truy vấn của M1 so baseline đã chọn ngưỡng là nhỏ: khoảng 0,2 điểm recall ở R1 và 0,4 điểm ở R1b theo số đã làm tròn. Không gán chênh lệch từ 36,8 lên 56,3 hoàn toàn cho M1, vì baseline một cấp đã đạt 55,9 nhờ thay cách chọn tham số. Coverage giảm khi thứ tự cụ thể và luật danh mục tập trung vào những sản phẩm có bằng chứng chung.

Trong nhóm truy vấn mà sản phẩm xem là hiếm của R1b, recall baseline một cấp là 16,7±18,0%, M1 là 37,7±18,3%. Kết quả hỗ trợ giả thuyết hierarchy có ích khi sản phẩm ít đơn, nhưng nhóm chỉ có khoảng 62 truy vấn/seed, SD lớn. Phải phân biệt “recall khi tiền đề hiếm” với “tỷ lệ sản phẩm hiếm từng được gợi ý ở hậu đề”: chỉ số thứ hai của M1 có thể thấp hơn baseline.

Chi phí tăng rõ: ở R1 support 0,002, độ dài giao dịch trung bình 1,82 → 3,43; itemset 169 → 1.770; Apriori 20,8 → 134,6 ms và FP-Growth 24,8 → 41,0 ms trong cặp đo M1. Số liệu lấy từ benchmark M1, không ghép 20,8 với 18,8 ở một lượt benchmark khác. Hiệu ứng mở rộng taxonomy làm tăng không gian mẫu và giải thích chi phí bổ sung.

6.10. Đánh giá M2

Bảng 6.5. M2 cải thiện tuân thủ ràng buộc nhưng giảm độ phủ catalog.

Hồ sơ/phương pháp	R@5 (%)	Coverage (%)	Hợp lý nghiệp vụ (%)
R1 – Một cấp	81,8 ± 0,3	89,8 ± 1,1	83,0 ± 1,0
R1 – M2	81,7 ± 0,2	73,2 ± 1,7	100,0 ± 0,0
R1 – M1+M2	81,9 ± 0,4	73,2 ± 1,7	100,0 ± 0,0
R1b – Một cấp	55,9 ± 1,0	86,3 ± 2,4	72,6 ± 1,0
R1b – M2	56,6 ± 0,7	69,2 ± 3,4	100,0 ± 0,0
R1b – M1+M2	56,9 ± 0,9	69,2 ± 3,4	100,0 ± 0,0

100% tuân thủ là hệ quả của dùng cùng các điều kiện A2–A4 để lọc và chấm. Số liệu không chứng minh tương thích kỹ thuật hoàn toàn. Recall không suy giảm đáng kể ở

các bộ giả lập này, nhưng không có bảo đảm cùng kết quả trên catalog khác, nơi tên sản phẩm thiếu chuẩn hoặc nhiều quan hệ hợp lệ bị quy tắc loại.

M2-B push/post cho cùng tập mục ở các cấu hình benchmark đã lưu; lợi ích tính toán phụ thuộc thuật toán. Trên R1b đa mức support 0,002, Apriori hậu lọc 103,9 ms và cắt tỉa 34,2 ms; candidate thực đếm giảm 3.790→639. FP-Growth cùng cấu hình hậu lọc 41,4 ms và push 53,4 ms: có cắt bớt cây nhưng overhead kiểm tra làm chậm hơn. Tỷ lệ "nhanh hơn" dưới 1 phải đọc là chậm hơn, không làm tròn thành một cải thiện.

6.11. Đánh giá M3

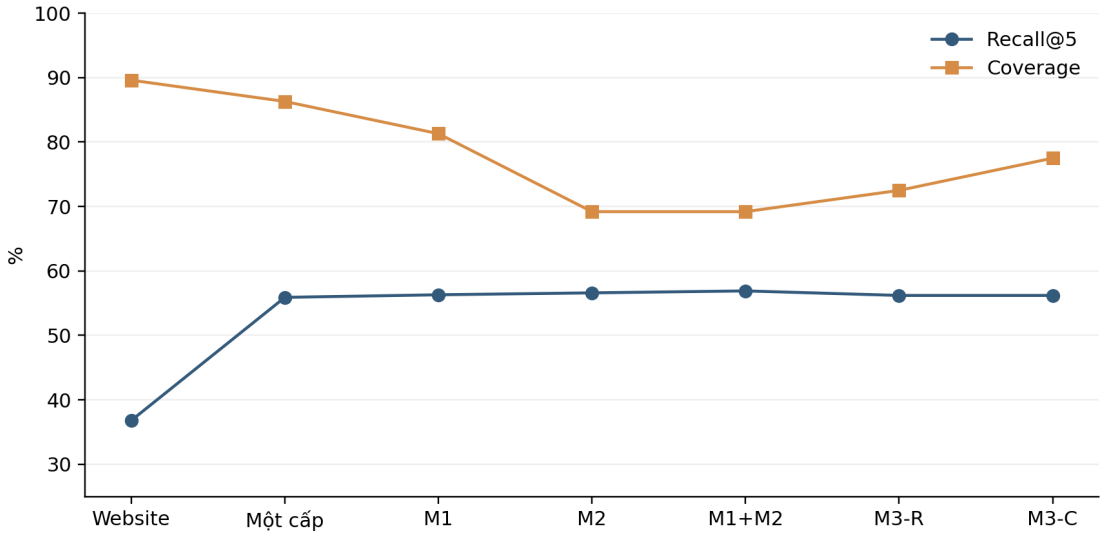
Bảng 6.6. Tác động xếp hạng và tiêu chí chọn tham số M3.

Hồ sơ/phương pháp	R@5 (%)	Hit@5 (%)	Coverage (%)	Đa dạng danh mục (%)
R1 – Một cấp	81,8 ± 0,3	84,1 ± 0,4	89,8 ± 1,1	66,7 ± 1,0
R1 – M3-R	82,3 ± 0,4	86,3 ± 0,8	86,3 ± 4,4	70,3 ± 6,5
R1 – M3-C	82,3 ± 0,3	85,9 ± 1,3	90,2 ± 0,0	65,2 ± 1,6
R1b – Một cấp	55,9 ± 1,0	63,1 ± 1,0	86,3 ± 2,4	62,2 ± 0,7
R1b – M3-R	56,2 ± 0,7	63,7 ± 0,3	72,5 ± 4,0	77,8 ± 7,1
R1b – M3-C	56,2 ± 0,7	63,7 ± 0,6	77,5 ± 9,0	72,1 ± 10,9

M3 có lợi ích nhỏ về recall, rõ hơn ở hit và đa dạng một số cấu hình. Trên R1b, M3-R tăng đa dạng nhưng giảm coverage hơn 13 điểm so baseline. Đa dạng đo số danh mục khác nhau chia độ dài từng danh sách; coverage đo số sản phẩm khác nhau được gợi ý trên toàn tập. Một danh sách đa dạng theo category vẫn có thể tập trung vào vài sản phẩm bán chạy lặp lại ở nhiều truy vấn.

Trên M1 hoặc M1+M2, validation thường chọn native và lambda=1; kết quả M3 gần như không đổi. Không nên mô tả mọi tổ hợp M1+M2+M3 là tốt hơn từng mô hình độc lập. Các phương pháp đều cần một tiêu chí chọn phù hợp và có thể tốt ở một mặt nhưng kém ở mặt khác.

Một kết quả cần đặc biệt thận trọng là recall nhóm hiếm 81,4±5,8% của M2+M3-C trên R1b. Báo cáo nghiên cứu phân tích toàn bộ mục cho máy hiếm trong tình huống này là fallback, không có luật. Tri thức tương thích theo tên lọc ra ốp đúng dòng, còn bộ sinh đã cài chính ốp đó làm companion. Vì vậy đây chủ yếu là lợi ích của tri thức và cấu trúc mô phỏng, không là bằng chứng miner khám phá quan hệ hiếm với recall 81,4%.



Hình 6.2. Recall và coverage trên R1b; số liệu đã lưu, không phải đo mới.

6.12. Thời gian chạy và tài nguyên

Thời gian cần phân biệt load dữ liệu, mining, sinh luật, xếp hạng, truy vấn bổ sung giá, truyền mạng và render. Trường meta.miningMs của service đo lần khai phá mô hình; ở cache hit nó vẫn là thời gian lần model được tạo, không phải thời gian request hiện tại. Dùng giá trị đó làm biểu đồ độ trễ API có cache sẽ gây hiểu nhầm.

RSS là bộ nhớ thường trú của tiến trình, không đồng nghĩa heap JavaScript hoặc toàn bộ bộ nhớ ứng dụng. Giá trị tăng RSS nhỏ/không dương có thể chịu ảnh hưởng GC và cấp phát hệ điều hành; không diễn giải dấu gạch trong bảng cũ thành 0 MB. Benchmark chưa bao gồm tải đồng thời, MongoDB, hai frontend và nhiều model cache online nên không thể suy từ RSS mining dưới khoảng vài chục MB thành cấu hình RAM production tối thiểu.

Nhánh nghiên cứu dùng tiến trình con và budget để giới hạn; API mặc định chưa dùng worker và budget đó. Hướng cải tiến thực tế là khai phá định kỳ nền, chỉ giữ luật $1 \rightarrow 1$ cần thiết, giới hạn model cache và phân phối phiên bản mô hình. Cần đo lại sau mỗi thay đổi, vì loại việc sinh luật không dùng có thể quan trọng hơn đổi thuật toán khi danh mục nhỏ.

6.13. Phân tích kết quả

Chọn ngưỡng trên validation là cải thiện lớn nhất trong dữ liệu đã lưu: recall R1 từ 64,3 lên 81,8%, R1b từ 36,8 lên 55,9% ở baseline một cấp. Đây là khác biệt về cấu hình và quy trình chọn tham số. M1 cải thiện nhóm hiếm nhờ khái quát hóa; M2 kiểm soát hướng gợi ý và heuristic tương thích; M3 cân bằng thứ tự và danh mục. Mỗi nhận định cần gắn đúng biến đổi đã thực hiện.

Các chỉ số hiện thực được xác định rõ như sau, với Q là tập truy vấn, G_q là truth, R_q là Top-K:

$$P@K = \frac{1}{|Q|}\sum_{q}\frac{|R_q \cap G_q|}{K}, \quad R@K = \frac{1}{|Q|}\sum_{q}\frac{|R_q \cap G_q|}{|G_q|}. \qquad (6.1)$$

$$Hit@K = \frac{1}{|Q|}\sum_{q}\mathbf{1}[|R_q \cap G_q| > 0], \quad Coverage = \frac{|\bigcup_q R_q|}{|I|}. \qquad (6.2)$$

Danh sách thiếu K vẫn chia K trong precision, nên mô hình rules-only bị phản ánh phần chỗ trống. Truth chỉ lấy từ đơn nhiều sản phẩm, làm giao thức tập trung vào đồng mua và bỏ đơn đơn lẻ. Việc lấy trung bình theo truy vấn khiến đơn có nhiều mục đóng góp nhiều truy vấn hơn; bootstrap theo đơn xử lý phụ thuộc nhưng không đổi trọng số định nghĩa chỉ số.

SD qua seed đo biến thiên của các lần sinh cùng cơ chế; khoảng tin cậy bootstrap theo đơn là một đối tượng thống kê khác. Không gọi một cột TB±SD là khoảng tin cậy 95%. Các bảng báo cáo gốc có so sánh cặp trên cùng truy vấn, phù hợp hơn so hai khoảng tin cậy độc lập bằng mắt. Các số làm tròn 0,1 điểm không đủ tự xác định ý nghĩa thống kê.

6.14. Hạn chế của thực nghiệm

Dữ liệu mô phỏng có quy luật cài sẵn và ít biến động theo thời gian; không có hiệu ứng giá, quảng cáo, thiếu hàng hoặc nhóm khách thay đổi như môi trường thật. Năm seed không đại diện năm cửa hàng độc lập. Nhóm hiếm nhỏ và tri thức tương thích phần nào trùng với cách sinh companion nên một số kết quả có thể lạc quan.

Hierarchy chỉ có hai cấp dữ liệu; thuật toán tổng quát hỗ trợ nhiều cấp nhưng chưa có bằng chứng hiệu quả trên taxonomy sâu. M2 suy từ tên, không từ thông số cổng, kích thước hay giao thức. M3 chỉ phạt category, chưa có tương đồng semantic và chưa tối ưu chung mọi tham số nền/ranking. Các chỉ số set-based không nhạy với thứ tự khi Top-5 giữ nguyên; chưa dùng NDCG/MRR trong đánh giá hiện tại.

Benchmark được đo trên một máy và lưu từ trước; phiên khảo sát không chạy lại toàn bộ 10 bộ benchmark để tự tạo một bộ số mới. Chưa có thử nghiệm tải, A/B test, tỷ lệ nhấp hoặc đơn phát sinh do gợi ý. Chưa có ảnh UI thực tế và kiểm chứng MongoDB đang triển khai. Những thiếu hụt này được đánh dấu để bổ sung, không dùng số liệu ước lượng thay thế.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết quả đạt được

Đối chiếu mã nguồn xác định TrungShop có các khối website cần thiết cho duyệt hàng, tài khoản, giỏ, tiếp nhận và quản lý đơn, coupon, bình luận và thống kê. Luồng gợi ý một cấp đã được nối từ component trang chi tiết qua API đến module mining. Hai thuật toán có hợp đồng chung, nhiều fixture đối chiếu sách và cơ chế nguồn gợi ý rõ ràng.

Nhánh nghiên cứu cài đặt M1, M2, M3 và bộ đánh giá theo thời gian. Kết quả lưu cho phép quan sát lợi ích của hierarchy đối với mục ít dữ liệu, ràng buộc đối với mức tuân thủ và ranking đối với đa dạng. Lần khảo sát còn tái hiện các ví dụ đầu ra từ một dataset có checksum xác định. Các kết quả này không làm thay đổi kết luận rằng M1–M3 chưa được gọi trong website mặc định.

7.2. Những đóng góp của đồ án

Đóng góp kỹ thuật là tách mining khỏi DB, dùng cùng chuẩn hóa/ngưỡng cho Apriori và FP-Growth, giữ chỉ mục luật để chọn theo sản phẩm, phối hợp cache snapshot và cache mô hình, phân biệt fallback trên API và UI. Đóng góp về hệ thống là không tin giá từ trình duyệt, bảo vệ đơn của tài khoản và giữ lịch sử giá/trạng thái.

Đóng góp nghiên cứu là áp dụng có kiểm soát các ý tưởng trong Chương 6–7 của giáo trình vào cấu trúc catalog hiện có, xây ràng buộc có tính chất toán học rõ, và khảo sát công thức ranking do đồ án lựa chọn. Giá trị của nghiên cứu nằm cả ở các kết quả không cải thiện: push có thể chậm FP-Growth, đa dạng có thể giảm coverage và M3 có thể không thêm lợi ích trên M1.

7.3. Những hạn chế

Các giới hạn chính là thiếu dữ liệu giao dịch thật và đánh giá kinh doanh, chưa tích hợp các mô hình nâng cao, chưa kiểm chứng UI/DB trong môi trường triển khai, chưa có kho định lượng hoặc cổng thanh toán. Cache trong tiến trình có khả năng cũ khi sửa đơn và không chia sẻ giữa nhiều instance; mining đồng bộ có rủi ro ảnh hưởng API khi dữ liệu hoặc số cấu hình tăng.

Ở mức mô hình, luật đồng mua không tự chứng minh tương thích hoặc cá nhân hóa. Taxonomy phẳng giới hạn chiều sâu M1; tri thức tên giới hạn M2; heuristic danh mục giới hạn M3. Các ưu điểm chỉ có ý nghĩa trong điều kiện dữ liệu và mục tiêu đánh giá đã khai báo, không nên mở rộng thành khẳng định hoàn thiện cho mọi cửa hàng.

7.4. Hướng phát triển

Ưu tiên đầu là chuẩn hóa tiêu chuẩn dữ liệu học: chọn trạng thái đơn được phép, xử lý đơn trùng/hủy, xác định khoảng thời gian và kiểm tra biến động. Thu thập dữ liệu thật có quyền sử dụng, chia thời gian và chọn lại tham số với baseline phù hợp. Chỉ sau đó mới quyết định mức mở rộng cần thiết; ngưỡng tốt nhất trên mô phỏng không tự động là ngưỡng triển khai.

Ưu tiên tiếp là chuyển mining sang tác vụ nền hoặc worker, giới hạn số mô hình cache, bổ sung version dữ liệu và kiểm tra race khi refresh. Cần chuẩn hóa price số và cải tiến query giá để tránh đọc toàn bộ ứng viên trong RAM. Các quy tắc tiền/đơn phải giữ tương thích lịch sử khi di trú schema.

Về nghiên cứu, nên xây ontology tương thích dựa trên thuộc tính cấu trúc, đánh giá độc lập bởi người có chuyên môn, rồi cân nhắc M2 cho cả luật và fallback. Chỉ dùng M1 khi dữ liệu thật đủ minh chứng nhu cầu khái quát hóa; M3 cần thêm chỉ số theo thứ hạng, kiểm tra tác động lên coverage và điều chỉnh lambda. Khi có hạ tầng, đánh giá A/B bằng tỷ lệ nhấp, thêm giỏ và mua, đồng thời giữ nhãn nguồn để phân tích đóng góp của luật so với fallback.

TÀI LIỆU THAM KHẢO

[1] Jiawei Han, Micheline Kamber, Jian Pei. *Data Mining: Concepts and Techniques*. Third Edition. Morgan Kaufmann/Elsevier, 2012. ISBN 978-0-12-381479-1. Bản PDF được cung cấp trong repository; năm 2012 theo trang bản quyền của bản này. Đối chiếu Chương 6: §6.2.1 Apriori, §6.2.2 sinh luật, §6.2.4 FP-Growth, §6.3.2–6.3.3 độ đo; Chương 7: §7.2.1 đa mức, §7.3.1 metarule, §7.3.2 ràng buộc, §7.5.2 top-k giảm dư thừa. Không gán số trang chưa đối chiếu cho trích dẫn.

[2] Rakesh Agrawal, Ramakrishnan Srikant. "Fast Algorithms for Mining Association Rules." *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, 1994. Tài liệu gốc của Apriori; báo cáo dùng [1] để đối chiếu chi tiết thuật toán và ví dụ.

[3] Jiawei Han, Jian Pei, Yiwen Yin. "Mining Frequent Patterns without Candidate Generation." *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2000. DOI: 10.1145/342009.335372. Công trình nền về FP-Growth.

[4] Ramakrishnan Srikant, Rakesh Agrawal. "Mining Generalized Association Rules." *Proceedings of the 21st International Conference on Very Large Data Bases (VLDB)*, 1995. Nền tảng khai phá quan hệ tổng quát qua hierarchy.

[5] React. *React Documentation*. https://react.dev/ — nguồn chính thức; phiên bản dependency của đồ án căn cứ package.json và lockfile, không suy từ phiên bản tài liệu hiện hành.

[6] Express. *Express 4.x API Reference*. https://expressjs.com/en/4x/api.html — router, middleware, request và response.

[7] MongoDB. *MongoDB Manual: Data Modeling*. https://www.mongodb.com/docs/manual/data-modeling/ — tài liệu nhúng và tham chiếu.

[8] Mongoose. *Mongoose 6 Documentation*. https://mongoosejs.com/docs/6.x/ — schema, model và truy vấn.

[9] Node.js. *Node.js Documentation*. https://nodejs.org/docs/latest-v20.x/api/ — runtime và test runner. Phiên bản thực nghiệm được ghi riêng theo log.

[10] M. Jones, J. Bradley, N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7519 — định dạng token; quy tắc role và thu hồi trong TrungShop là quyết định ứng dụng.

[11] Repository *Product-Recommendation*. Snapshot khảo sát HEAD `a9c5a23280b695467aac045a36079c9dbf543163`. Các phần `Shop/Shop/backendapi/src`, `research`, `scripts`, `test`, `research-results`, `Shop/Shop/frontend/src` và `Admin/Admin/src`. Mã nguồn là bằng chứng hiện thực, không thay thế tài liệu lý thuyết.

Các đường dẫn tài liệu công nghệ trên là nguồn chính thức để tra cứu; nội dung phiên khảo sát được đối chiếu trực tiếp với mã và bản sách cục bộ, chưa kiểm tra trực tuyến toàn văn từng trang tài liệu công nghệ. Khi hoàn thiện theo mẫu trường, sinh viên cần thống nhất chuẩn trích dẫn và bổ sung ngày truy cập thực tế nếu yêu cầu.

PHỤ LỤC A. BẢN ĐỒ MÃ NGUỒN

Phạm vi kiểm kê gồm 260 tệp JS/JSX/CSS ngoài tài nguyên public. Manifest SHA-256 ghi rõ đường dẫn, số dòng và tên hàm nhận diện tĩnh; việc đếm tệp không tự chứng minh mọi nhánh đã chạy. Các đường trong bảng dưới tương đối với backend.

Bảng A.1. Bản đồ module trọng tâm.

Nhóm	Tệp	Trách nhiệm
Mining	src/apps/services/mining/transactions.js	Chuẩn hóa giao dịch và ngưỡng
Mining	src/apps/services/mining/apriori.js	Join/prune/count theo mức
Mining	src/apps/services/mining/fpgrowth.js	FP-tree và cây điều kiện
Mining	src/apps/services/mining/rules.js	Sinh luật và độ đo
Gợi ý	src/apps/services/recommendation/service.js	Cache và điều phối
Gợi ý	src/apps/services/recommendation/ranking.js	Tầng 1, lọc và fallback
Gợi ý	src/apps/services/recommendation/hydrator.js	Dựng order/product/fingerprint
M1	research/multilevel/hierarchy.js	Cây phân cấp, mở rộng giao dịch
M1	research/multilevel/mining.js	Ngưỡng cấp, lọc tầm thường, dư thừa
M1	research/multilevel/recommender.js	Ánh xạ danh mục về sản phẩm
M2	research/constraints/constraints.js	Phân loại ràng buộc
M2	research/constraints/business.js	Heuristic nghiệp vụ
M2	research/constraints/miners.js	Push/post
M2	research/constraints/recommender.js	Ghép ràng buộc với gợi ý
M3	research/ranking/strategies.js	Điểm và Top-K đa dạng
M3	research/ranking/recommender.js	Factory phối hợp nền
Đánh giá	research/evaluation/split.js	Chia theo thời gian
Đánh giá	research/evaluation/protocol.js	Một sản phẩm xem, phần còn lại truth
Đánh giá	research/evaluation/metrics.js	Độ đo và bootstrap theo đơn
Dữ liệu	research/datasets/generator.js	R1/R1b, PRNG, checksum

PHỤ LỤC B. DANH SÁCH ROUTE THEO ROUTER

Bảng được trích từ router hiện hành. Quyền của các nhóm có router.use được ghi riêng; không suy quyền từ regex trích route. Prefix mặc định `/api/v1`.

Bảng B.1. Route khai báo trong các tệp router.

Router	Phương thức	Đường con	Handler/middleware trong khai báo
auth.js	POST	`/login`	`loginLimit, asyncHandler(AuthController.login))`
auth.js	POST	`/register`	`registerLimit, asyncHandler(AuthController.register))`
auth.js	GET	`/me`	`requireAuth, asyncHandler(AuthController.me))`
auth.js	PUT	`/me`	`requireAuth, asyncHandler(AuthController.updateMe))`
auth.js	PUT	`/me/password`	`requireAuth, asyncHandler(AuthController.changePassword))`
category.js	POST	`/create`	`requireAdmin, asyncHandler(CategoryController.create))`
category.js	PUT	`/update/:id`	`requireAdmin, asyncHandler(CategoryController.update))`
category.js	DELETE	`/delete/:id`	`requireAdmin, asyncHandler(CategoryController.delete))`
category.js	GET	`/`	`asyncHandler(CategoryController.index))`
category.js	GET	`/:id`	`asyncHandler(CategoryController.show))`
category.js	GET	`/:id/products`	`asyncHandler(ProductController.getProductsByCategoryId))`
comment.js	GET	`/admin`	`asyncHandler(CommentController.list))`
comment.js	DELETE	`/admin/:id`	`asyncHandler(CommentController.remove))`

Router	Phương thức	Đường con	Handler/middleware trong khai báo
coupon.js	POST	`/preview`	`couponPreviewLimit, asyncHandler(OrderController.previewCoupon))`
coupon.js	GET	`/admin`	`requireAdmin, asyncHandler(CouponController.list))`
coupon.js	POST	`/admin`	`requireAdmin, asyncHandler(CouponController.create))`
coupon.js	GET	`/admin/:id`	`requireAdmin, asyncHandler(CouponController.show))`
coupon.js	PUT	`/admin/:id`	`requireAdmin, asyncHandler(CouponController.update))`
coupon.js	PATCH	`/admin/:id/status`	`requireAdmin, asyncHandler(CouponController.setStatus))`
order.js	POST	`/`	`optionalAuth, asyncHandler(OrderController.order))`
order.js	GET	`/my`	`requireAuth, asyncHandler(OrderController.myOrders))`
order.js	GET	`/getAll`	`requireAdmin, asyncHandler(OrderController.getAllOrders))`
order.js	GET	`/topProductPairs`	`requireAdmin, asyncHandler(OrderController.getTopProductPairsNames))`
order.js	GET	`/admin`	`requireAdmin, asyncHandler(OrderController.adminList))`
order.js	GET	`/admin/:id`	`requireAdmin, asyncHandler(OrderController.adminDetail))`
order.js	PATCH	`/admin/:id/status`	`requireAdmin, asyncHandler(OrderController.updateStatus))`
product.js	POST	`/create`	`requireAdmin, asyncHandler(ProductController.addProduct))`

Router	Phương thức	Đường con	Handler/middleware trong khai báo
product.js	POST	`/upload-image`	`requireAdmin, ProductController.uploadImage)`
product.js	GET	`/`	`asyncHandler(ProductController.index))`
product.js	GET	`/:id`	`asyncHandler(ProductController.show))`
product.js	GET	`/:id/comments`	`asyncHandler(ProductController.comments))`
product.js	POST	`/:id/comments`	`requireAuth, asyncHandler(ProductController.storeComments))`
product.js	DELETE	`/:id/comments/:commentId`	`requireAdmin, asyncHandler(ProductController.deleteComment))`
product.js	PUT	`/:id/update`	`requireAdmin, asyncHandler(ProductController.updateProduct))`
product.js	DELETE	`/:id/delete`	`requireAdmin, asyncHandler(ProductController.deleteProduct))`
product.js	GET	`/:id/product-suggestions`	`asyncHandler(ProductSuggestionsController.getProductSuggestions))`
recommendation.js	GET	`/products/:id`	`controller.forProduct)`
recommendation.js	GET	`/status`	`controller.status)`
recommendation.js	POST	`/refresh`	`controller.refresh)`
stats.js	GET	`/summary`	`asyncHandler(StatsController.summary))`
stats.js	GET	`/revenue`	`asyncHandler(StatsController.revenue))`
stats.js	GET	`/top-products`	`asyncHandler(StatsController.topProducts))`
user.js	POST	`/create`	`asyncHandler(UserController.create))`

Router	Phương thức	Đường con	Handler/middleware trong khai báo
user.js	PUT	`/update/:id`	`asyncHandler(UserController.update))`
user.js	DELETE	`/delete/:id`	`asyncHandler(UserController.delete))`
user.js	GET	`/`	`asyncHandler(UserController.index))`
user.js	GET	`/:id`	`asyncHandler(UserController.getById))`

Các router user, comment và stats áp requireAdmin toàn nhóm; các router còn lại có quyền theo từng route. Recommendation status công khai; refresh giới hạn peer loopback trong controller. Các đường con phải ghép với mount trong web.js.

PHỤ LỤC C. NHẬT KÝ KIỂM CHỨNG

Ngày khảo sát: 02/11/2026 (UTC+7). Snapshot HEAD `a9c5a23280b695467aac045a36079c9dbf543163`. Các tệp nguồn không bị sửa để viết báo cáo; dependency được cài để chạy test.

Lệnh kiểm thử thành công: `/workspace/work/runtime/node_modules/node/bin/node --test test/*/*.test.js` tại backendapi, Node v20.20.2. Kết quả: 264 tests, 263 pass, 0 fail, 1 skipped. Test bị bỏ qua cần biến RECO_DB_IT=1 và MongoDB GR2 có dữ liệu; phiên khảo sát không chạy test DB thật. Log đầy đủ nằm trong bang_chung/test-node20.txt.

Lần kiểm thử Node v24.19.0 gặp lỗi Utils.isRegExp trong config. Đây là giới hạn tương thích runtime; log được giữ riêng, không thay kết quả Node 20. Các ví dụ thuần mining/M1/M2/M3 chạy được với Node 24 và được ghi trong examples.json. Chưa chạy bộ test React hoặc trình duyệt E2E.

Không chạy lại toàn bộ benchmark để tạo số thời gian mới. Các bảng Chương 6 lấy từ artifact cũ; ngày tạo, runtime và cách đo được giữ trong JSON/Markdown gốc. Script examples.js ghi minh họa từ R1b/20261101, không ghi MongoDB và không được dùng như benchmark.

PHỤ LỤC D. CÁC BƯỚC TÁI LẬP

Cài dependency backend theo lockfile bằng npm ci; dùng Node 20 để tương thích dependency config hiện tại. Chạy test bằng mẫu glob tệp, không truyền tên thư mục cho Node 24 rồi giả định test runner tự tìm cùng cách. Không cần khóa JWT thật hoặc kết nối DB cho các test offline/fake-app đã có cấu hình thử nghiệm.

Các lệnh nghiên cứu: `node scripts/benchmark-research-datasets.js`; `node scripts/evaluate-research-datasets.js`; `node scripts/benchmark-multilevel.js`; `node scripts/evaluate-multilevel.js`; `node scripts/benchmark-constraints.js`; `node scripts/evaluate-constraints.js`; `node scripts/evaluate-ranking.js`. Mỗi script có thể ghi lại research-results; sao lưu artifact gốc trước khi tái lập để không lẫn kết quả hai môi trường.

Có thể chạy riêng `--profile r1b --seed 20261101`. Không chạy seed-demo ghi DB chỉ để tái lập thống kê offline. Đọc hướng dẫn và safety guard của script nếu cần tạo một DB demo riêng. Tệp examples.js có đường dẫn tuyệt đối của môi trường khảo sát; khi chuyển máy, sửa biến B về thư mục backend tương ứng.

PHỤ LỤC E. DỮ LIỆU VÍ DỤ MỚI

Dataset R1b/20261101 có checksum `ac2ea12f86d60fe2c435b4a72ce9552d29e5cf99`, khớp manifest đã lưu. Lịch sử dùng 16000 đơn; số đếm singleton và đầu ra dưới đây được chạy trong phiên khảo sát. Các ví dụ là dữ liệu giả lập; không chứa thông tin cá nhân khách hàng.

E.1. Samsung Galaxy S24 FE 128GB

ID: `ff2edeb364b264fd9bad99fb`; số đơn lịch sử chứa sản phẩm: 51.

Bảng E.1. Đầu ra minh họa Samsung Galaxy S24 FE 128GB.

Mô hình	Vị trí	Sản phẩm	Nguồn	Điểm nếu có
flat	1	Samsung Galaxy S24 Ultra 256GB	fallback	—
flat	2	Samsung Galaxy A05s 128GB	fallback	—
flat	3	Samsung Galaxy S23 Ultra 256GB	fallback	—
flat	4	Samsung Galaxy M34 5G 128GB	fallback	—
flat	5	Samsung Galaxy S24+ 256GB	fallback	—
m1	1	Củ sạc nhanh Samsung 45W	association-rule	0.144024
m1	2	Ốp lưng Galaxy S24 Ultra	association-rule	0.124966
m1	3	Kính cường lực Galaxy S24 Ultra	association-rule	0.089573
m1	4	Tai nghe có dây Samsung IA500	association-rule	0.087122
m1	5	Ốp lưng trong suốt iPhone 14 series	association-rule	0.024231
m2	1	Củ sạc nhanh USB-C 20W	fallback	—
m2	2	Pin dự phòng Xiaomi 10.000mAh 20W	fallback	—
m2	3	Tai nghe Apple AirPods 2	fallback	—
m2	4	Pin dự phòng Samsung 20.000mAh 45W	fallback	—

Mô hình	Vị trí	Sản phẩm	Nguồn	Điểm nếu có
m2	5	Cáp Apple USB-C to USB-C 1m	fallback	—
m3	1	Samsung Galaxy S24 Ultra 256GB	fallback	—
m3	2	Củ sạc nhanh USB-C 20W	fallback	—
m3	3	Ốp lưng silicon iPhone 15	fallback	—
m3	4	Pin dự phòng Xiaomi 10.000mAh 20W	fallback	—
m3	5	Tai nghe Apple AirPods 2	fallback	—

E.2. iPhone 15 128GB

ID: `1d5ba7380b7670883116b85e`; số đơn lịch sử chứa sản phẩm: 1146.

Bảng E.2. Đầu ra minh họa iPhone 15 128GB.

Mô hình	Vị trí	Sản phẩm	Nguồn	Điểm nếu có
flat	1	Ốp lưng silicon iPhone 15	association-rule	—
flat	2	Kính cường lực iPhone 15	association-rule	—
flat	3	Củ sạc nhanh USB-C 20W	association-rule	—
flat	4	Cáp Apple USB-C to USB-C 1m	association-rule	—
flat	5	Tai nghe Apple AirPods 2	association-rule	—
m1	1	Ốp lưng silicon iPhone 15	association-rule	0.415358
m1	2	Kính cường lực iPhone 15	association-rule	0.286213
m1	3	Củ sạc nhanh USB-C 20W	association-rule	0.226876
m1	4	Cáp Apple USB-C to USB-C 1m	association-rule	0.183246
m1	5	Tai nghe Apple AirPods 2	association-rule	0.096859
m2	1	Ốp lưng silicon iPhone 15	association-rule	—
m2	2	Kính cường lực iPhone 15	association-rule	—
m2	3	Củ sạc nhanh USB-C 20W	association-rule	—
m2	4	Cáp Apple USB-C to USB-C 1m	association-rule	—
m2	5	Tai nghe Apple AirPods 2	association-rule	—
m3	1	Ốp lưng silicon iPhone 15	association-rule	0.371460
m3	2	Cáp Apple USB-C to USB-C 1m	association-rule	0.184465
m3	3	Kính cường lực iPhone 15	association-rule	0.307393
m3	4	Tai nghe Apple AirPods 2	association-rule	0.090988
m3	5	Củ sạc nhanh USB-C 20W	association-rule	0.171247